

David McIntosh

Using Bayesian Networks to Simulate Bias in Algorithmic Recruiting

Computer Science Tripos - Part II

Churchill College
2025

Declaration of Originality

I, the candidate for Part II of the Computer Science Tripos with Blind Grading Number 2450A, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose.

In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy.

I am content for my report to be made available to the students and staff of the University.

May 17, 2025

Proforma

Candidate Number: 2450A
Project Title: Using Bayesian Networks to Simulate Bias in Algorithmic Recruiting
Examination: Computer Science Tripos - Part II
Year: 2025
Dissertation Word Count: 11879
Code Line Count: 8696
Project Originator: The candidate
Project Supervisor: David Khachaturov (dgg27)

Original Aims of the Project

The aim of the project is to investigate bias within a rigorously motivated simulated recruitment scenario.

Each run of the simulation, a random bayesian network is generated as an applicant distribution, and machine-learning recruiters make hiring decisions on sampled applicants. The bias they exhibit on those decisions is quantified. This process is systematically repeated across varying recruiter models, application generation conditions, and post-training bias mitigations. The data produced is analysed to assess and characterise algorithmic bias in ML classification processes.

Work Completed

All requirements were completed, and a full visualisation and walkthrough was produced for the research at modelling-bias.com (link is anonymised).

Having developed a theoretical framework for investigating bias, I discover the following under sensible assumptions:

- Bias often emerges, even with an unbiased training set.
- Standard fairness metrics often disagree.
- Bias tends to exacerbate existing group differences.
- Masking protected characteristics reduces bias.
- Model choice affects bias.
- Post-training mitigations reduce bias to an extent.

Each of these findings either contradicts existing practice or otherwise has significant implications for fairness within machine learning.

Both the research and implementation were critically evaluated.

Special Difficulties

None.

Table of Contents

1	Introduction	1
1.1	Motivation	1
1.2	Related Work	2
2	Preparation	3
2.1	Starting Point	3
2.2	Preliminaries	3
2.2.1	Fairness Criteria	3
2.2.2	Comparison between Criteria	4
2.2.3	Bayesian Networks	5
2.3	Simulation Design	6
2.3.1	Generating the Applicant Distribution	7
2.3.2	A Run of the Simulation	9
2.4	Tools Used	10
2.5	Proposal Refinement and Requirements	12
2.6	Software Methodology	13
2.7	Ethical Considerations	13
3	Implementation	14
3.1	Generating a Bayesian Network	14
3.2	Generating Applicants and Applications	16
3.3	Recruiters	18
3.4	Measuring Bias	19
3.5	Applying Post-Training Bias Mitigations	19
3.6	Experimenting with the Simulation	21
3.7	Visualisation and Walkthrough	21
3.8	Website Demonstration	22
3.9	Repository Overview	25
4	Evaluation	28
4.1	Results and Findings	28
4.1.1	1. Bias Often Emerges, even with an Unbiased Training Set	29
4.1.2	2. Standard Fairness Metrics often Disagree	30
4.1.3	3. Bias Tends to Exacerbate Existing Group Differences	31
4.1.4	4. Masking Protected Characteristics Reduces Bias	32
4.1.5	5. Model Choice Affects Bias	33
4.1.6	6. Post-Training Mitigations Reduce Bias to an Extent	34
4.2	Assumptions Analysis	35
4.3	Research Implications	36
4.4	Evaluation against Requirements	38
5	Conclusions	39
5.1	Summary of Implications and Recommendations	39
5.2	Further Work	39
5.3	Personal Lessons Learnt	40

A	Appendices	48
A.1	Assumed Preliminaries	48
A.1.1	Binary Classification Terminology	48
A.1.2	Directed Acyclic Graphs	48
A.2	Calculating $c_{m,s,k}$	49
A.3	An ML Task Where Predictive Parity is a Suitable Metric	50
A.4	Recruiter Specifications	51
A.5	Conceptualising Bias Metrics	54
A.6	Data Analysis Example	55
A.7	Assumptions Analysis	57
A.8	Tools Used for Writing Document	58
B	Project Proposal	59

1 Introduction

1.1 Motivation

Machine learning (ML) is becoming increasingly prevalent in decision-making, finding use in resume screening,^{[1][2]} predicting crime and recidivism,^{[3][4]} loan approval,^[5] and medical diagnoses.^[6] It influences the products we buy,^[7] the entertainment recommended to us,^[8] and the information we see on search engines.^[9] Existing research has shown that many modern ML systems exhibit bias,^{[10][11]} such that the decisions they make are unfair towards certain groups.

For ease of communication and conceptualisation, I narrow our focus to recruitment, whereby an ML recruiter classifies applicants into the classes “hire” or “reject”, based on their applications. If I were to investigate this scenario using real data, I would be likely to suffer three difficulties:

1. It is difficult to gather sufficient data to assess bias across many decision-making scenarios.
2. Some metrics for bias require some notion of ground truth^[12] (how competent or meritorious an applicant actually is), but this is influenced by who measures it.
3. The training set is likely to be biased, so it is difficult to isolate the bias which emerges from the recruiter’s decision-making.

By using simulated data, I avoid these problems. I can quickly synthesise a sufficiently-sized dataset, and ground truth can be explicitly generated. However, this approach presents an additional requirement. The simulated data needs to be drawn from a distribution which approximates that of reality. A bayesian network is a model for a joint probability distribution which encodes causality between variables.^[13] I justify why this is a suitable distribution for applicants in Section 2.3.1.

A standard causal explanation for machine unfairness is that bias in a training set is propagated to bias in a model’s output.^{[14][15]} This would imply that bias can be fully mitigated by debiasing the training dataset,^{[16][17]} and that a model is only as biased as its input data. In this dissertation, I show that this is an incomplete interpretation. Bias can exist when training datasets are fully representative of the underlying true distribution (Result 4.1.1).

Additionally, I demonstrate that two standard fairness criteria exhibit high disagreement in practice (Result 4.1.2), a problem compounded by the fact that many existing systems use an unsuitable fairness metric.^{[18][19]} Given then that this dissertation offers a robust criticism of current working practices, its contributions have the potential to meaningfully inform and improve the field.

I investigate these findings and related phenomena by measuring bias across many simulations in which a recruiter makes hiring decisions on sampled applicants, systematically varying recruiter models, application generation conditions, and post-training bias mitigations. As with any simulation, the value of these results depends on whether they extrapolate to real-world recruiting. I evaluate this by analysing each assumption made in the simulation’s design.

While I focus on recruiting, this largely doesn’t affect the implementation of the simulation. To an extent, the results generalise across the field of ML classification.

Finally, I think the findings presented in this dissertation are valuable to a wider audience, including one of policy-makers, recruiters and casual readers. However, they require some context to fully understand. Therefore, I also developed a website to host a visualisation and walkthrough of the research at modelling-bias.com. Markers are strongly encouraged to visit this website.

1.2 Related Work

Fairness in machine learning is an emerging field,^[20] although research exists as far back as 2002.^[21] There are numerous examples of bias in ML systems. For example, the UK passport photo checker exhibits bias against dark-skinned women,^[22] a healthcare algorithm used to identify high-risk patients was less likely to identify black patients for extra care,^[23] and facial-detection systems show differences in accuracy between protected groups.^[24]

Human recruiting is widely agreed to exhibit bias.^{[25][26][27][28]} In 2003, Bertrand and Mullainathan found that white names receive 50 per cent more call-backs for a response to a job posting.^[29] Veldanda et al. research whether that effect is replicated when LLMs are used to classify applicants.^[30] They found no detectable bias on race and gender, although bias did exist on the grounds of political affiliation.

Machine learning has seen widespread use in recruiting for some time,^{[31][32]} and this is predicted to increase. Although bias is expected to exist as in other domains, fewer concrete examples of bias in ML recruitment exist.^[33] One example is Amazon stopping the use of an internal resume screening tool thought to disadvantage women.^[34]

I use terminology and draw largely from the textbook *Fairness and Machine Learning* (Barocas et al.).^[12] I also make significant use of bayesian networks, first motivated by Pearl.^[13]

Simulation studies have precedent in epidemiology,^{[35][36]} but no existing research has applied the approach to investigate unfairness.

COMPAS Case Study

This dissertation was influenced by the now infamous disagreement between Northpointe and journalism organisation ProPublica, concerning the former's COMPAS tool for predicting recidivism^[37] among inmates. ProPublica found the tool exhibited bias. Black defendants who did not reoffend were nearly twice as likely as white defendants to be misclassified as high risk.^[38]

Northpointe published a response claiming the tool was not racially biased because the same proportion of both groups were falsely classified as high-risk, and the same proportion were falsely classified as low-risk.^[39]

In this instance, there was high disagreement between two established criteria for bias (equalised odds and predictive parity). In this dissertation, I will demonstrate that this disagreement is common under certain conditions (Result 4.1.2).

2 Preparation

2.1 Starting Point

For a talk I gave in second year, I had researched bayesian networks, algorithmic recruitment, and machine fairness. I had written no code, I knew little of the technical aspects for how to create the simulation, and I had limited theoretical knowledge. I had correctly anticipated that this was a high-risk dissertation involving much effort and thought. I detail my initial experience with each tool in Section 2.4.

2.2 Preliminaries

As I use it, **Machine Learning** (ML) refers to any decision-making system which doesn't rely on domain-specific rules, instead learning to replicate patterns in a training set.

I assume that the intention of a recruiter is to hire the most meritorious applicant, so I use the terms **Competence**, **Meritoriousness** and **Qualifiedness** interchangeably to refer to the ground truth for each applicant, a concept needed for some of these criteria.

Assuming standard binary classification terminology, defined in Appendix A.1.1, we have that:

- **False Negative Rate** (FNR) is the proportion of qualified applicants who were rejected.
- **False Positive Rate** (FPR) is the proportion of unqualified applicants who were hired.
- **False Omission Rate** (FOR) is the proportion of rejected applicants who were qualified.
- **False Discovery Rate** (FDR) is the proportion of hired applicants who were unqualified.

I use the terms **Bias** and **Unfairness** interchangeably to refer to the unequal treatment of a particular group.

I restrict the scope of this dissertation to **Output Bias**,^[10] bias that can be detected and measured based only on the decisions made, not how they were made or what information was used. Even then, this definition is ambiguous. I formalise it under established criteria, borrowing terminology from the *Fairness in Machine Learning*^[12] textbook, and applying it directly to recruiting.

Fairness Criteria

A recruiter which satisfies **Demographic Parity** is one for which the hiring rate is the same across groups. The same proportion of group A and group B are hired. Equivalently, a random applicant from group A has the same chance of being hired as a random applicant from group B.

A recruiter which satisfies **Equalised Odds** has the same false negative rate and false positive rate across groups.^[40] Equivalently, a random qualified applicant from group A is just as likely to be hired as a random qualified applicant from group B, and a random unqualified applicant from group A is just as likely to be hired as a random unqualified applicant from group B.

A recruiter which satisfies **Predictive Parity** has the same false discovery rate and false omission rate across groups.^[41] Equivalently, of applicants who were hired, the same proportion of group A and group B are unqualified, and of applicants who were rejected, the same proportion of group A and group B are qualified.

Fairness Criterion	Condition	Interpretation
Demographic Parity	Equal Hiring Rates across groups	The proportion of applicants hired is the same across groups. A random applicant from group A is as likely to be hired as one from group B.
Equalised Odds	Equal FPR and FNR across groups	Qualified applicants and unqualified applicants have equal chances of being hired, regardless of group.
Predictive Parity	Equal FDR and FOR across groups	Among those hired, the proportion who are unqualified is the same across groups. Among those rejected, the proportion who are qualified is the same across groups.

In any nontrivial classification task, these three criteria are mutually unsatisfiable.^[12] That is, no more than one of the criteria may be satisfied. The extent to which they differ empirically has not yet been rigorously studied.

If we denote whether an applicant is hired as the random variable $\hat{Y}$, whether an applicant is competent as Y , and the applicant's protected characteristic as Z (we motivate this terminology in the next section), we can formulate these criteria in terms of independencies between variables:^[12]

Fairness Criterion	Independence Condition
Demographic Parity	$Z \perp \hat{Y}$
Equalised Odds	$Z \perp \hat{Y} \mid Y$
Predictive Parity	$Z \perp Y \mid \hat{Y}$

Comparison between Criteria

Any preference between demographic parity or equalised odds will depend on the notion of what it means for a process to be fair.

A justification for demographic parity is that if group A and group B don't carry inherent differences, then any differences in merit between the two groups are a result of past unfair disadvantages, so a fair system should act as it would if those disadvantages didn't exist.^[12] The decisions made by such a system will usually partially mitigate those disadvantages. Equalised odds is perhaps more intuitive: an applicant's competence should determine whether they are hired or not, not their group membership.

My own opinion is that predictive parity is an inadequate criterion for fairness in the domain of recruiting. Intuitively, it appears to be similar to equalised odds, but I find that the two metrics exhibit high disagreement (Result 4.1.2). If the person who suffers most from hiring discrimination is the applicant, then it is sensible to judge fairness in terms of how an applicant's treatment is expected to change due to group membership, as in equalised odds. Predictive parity is instead how an external observer may try to approximate how unfair a process is. I consider it to be the appearance of bias, but it doesn't address an individual's treatment.

In some domains, predictive parity is the more reasonable metric. I provide an example in Appendix A.3.

Bayesian Networks

A bayesian network is a model $\mathcal{B} = (G, \Phi)$ to represent a system of dependent random variables:^[13]

- $G = (V, E)$ is a DAG (defined in Appendix A.1.2) in which each node $v \in V$ represents a random variable and each edge $v_i \rightarrow v_j$ represents a conditional dependency of v_j on v_i .
- Φ is a set of probability distributions. For each random variable $A \in V$, the conditional probability distribution in $P(A | \text{Parents}(A))$ is specified in Φ . If A has no parents, $P(A)$ is a prior distribution.

A bayesian network (BN) uniquely defines the joint distribution over all its variables as:

$$P_{\mathcal{B}}(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i | \text{Parents}(X_i))$$

It implies a constraint on that joint distribution, the Markov property:^[42] each variable is conditionally-independent of its non-descendants given its parents. While a bayesian network can denote any DAG, I constrain focus to networks denoting weakly connected DAGs.

An interactive and applied explanation of bayesian networks is in the walkthrough:

modelling-bias.com/walkthrough/how_do_we_simulate_candidates

2.3 Simulation Design

The standard model of recruitment is one of a recruiter making binary hiring decisions on applicants, based on their applications. For clarity, I formally define each of the components involved. This section would otherwise be included in the implementation, but the remainder of this chapter relies on theory developed here.

A **Characteristic** is a categorical random variable C_i with domain $\mathcal{C}_i$, representing some attribute of a person. This could represent the score a person received on an interview, their eye colour, etc. There is no constraint for any characteristic to be relevant to a recruiter. Let $\mathbf{C} = (C_0, C_1, \dots, C_n)$ be the **Characteristic Vector** and $\mathcal{C} = \mathcal{C}_0 \times \dots \times \mathcal{C}_n$ be the domain of the vector.

The **Protected Characteristic** is a categorical random variable $Z = C_z$. This models whether an applicant is of some protected group, or which group that they belong to (e.g. - what gender, race, age, sexuality, or religion). The **Competence Characteristic** is a binary random variable $Y = C_y$ where $\mathcal{C}_y = \{\text{competent}, \text{not competent}\}$. This models qualifiedness, whether the applicant should be hired by a fair recruiter.

An **Applicant** is an instantiation of the characteristic vector $\mathbf{c} = (\mathbf{c}_0, \mathbf{c}_1, \dots, \mathbf{c}_n)$, including an instantiation $\mathbf{z} = \mathbf{c}_z$ of the protected characteristic and an instantiation $\mathbf{y} = \mathbf{c}_y$ of the competence characteristic. The **Applicant Distribution** $\mathcal{D}$ is the joint distribution over the characteristic vector, such that $\mathbf{C} \sim \mathcal{D}$. Each applicant is then an independent sample of the applicant distribution.

An **Application** is a subvector $\mathbf{x} = (\mathbf{c}_i : i \in \mathcal{A})$ of an applicant, where $\mathcal{A} \subseteq \{1, \dots, n\} \setminus \{y\}$ is a fixed index set which excludes the competence characteristic. The corresponding random variable is denoted $X = (C_i : i \in \mathcal{A})$. Within a simulation run, $\mathcal{A}$ is fixed across all applicants. In real-world recruiting, these applications may be learnt from a resume-screening process or an interview.

A **Recruiter** R is an ML classifier which learns a function $h_\theta : \Pi_{i \in \mathcal{A}} \mathcal{C}_i \rightarrow (0, 1)$ which maps an application to a score, and a decision rule that maps scores to hiring decisions:

$$d_{\mathbf{x}_1, \dots, \mathbf{x}_n} : (0, 1) \rightarrow \{\text{hire}, \text{reject}\}$$

θ represents the model's parameters. The recruiter makes a hiring decision $\hat{y}$ for each applicant based on their application such that $\hat{y} = d_{\mathbf{x}_1, \dots, \mathbf{x}_n}(h_\theta(\mathbf{x}))$. During training, the model receives a dataset $D_{\text{train}} = \{(\mathbf{x}^{(j)}, \mathbf{y}^{(j)})\}_{j=0}^m$ consisting of applications and associated competences. The training process is unconstrained, but the model is intended to learn to hire applicants who are competent and reject applicants who aren't.

A recruiter isn't modelled as returning a binary decision from each application, but rather a score in the interval $(0, 1)$. The recruiter's default decision rule $d_{\mathbf{x}_1, \dots, \mathbf{x}_n}$ is:

$$\hat{y} = \begin{cases} \text{hire} & \text{if } h_\theta(\mathbf{x}) \geq h \\ \text{reject} & \text{if } h_\theta(\mathbf{x}) < h \end{cases}$$

where h is a threshold chosen such that the proportion of hired applicants in the testing set is the same as the proportion of competent applicants in the training set. This models how recruiters are likely to fill a quota when hiring. A **Post-Training Bias Mitigation** is a modification to a recruiter's decision rule.

A **Bias Measurement** is a function:

$$b : \{\text{hire}, \text{reject}\}^N \times \{\text{competent}, \text{not competent}\}^N \times \mathcal{C}_y^N \rightarrow \mathbb{R}$$

mapping the decisions made by a recruiter, the applicants' competence, and the applicants' protected characteristics to some score representing the degree of unfairness.

Generating the Applicant Distribution

The distribution $\mathcal{D}$ from which applicants are drawn would ideally meet the following requirements:

- It models that of the real world.
- It can be randomly generated in a performant manner.
- It is intuitive and explainable.

Typically, we have that $\neg(Y \perp X)$ and $\neg(Y \perp Z)$. The competence characteristic is neither independent of the application nor the protected characteristic. Many human characteristics are likely to be correlated to some extent, because they are shaped by the same influences, and any typical dataset contains many features slightly correlated with a protected attribute.^[12]

To explain why the competence characteristic is correlated with a protected characteristic, we instinctively avoid explanations ascribing it to a direct causal dependency, as in “this person is less qualified because they belong to a protected group”. The relationship is more likely to be indirect and brought about by a series of conditional dependencies.^{[43][44]}

For example, one such explanation is that membership of a protected group is historically related to economic disadvantage and societal bias, which in turn affects access to education and professional networks. Those factors, in turn, affect the development of relevant skills.

Any specific explanation of that sort will be incomplete and reductive. Nevertheless, if we assume that the protected characteristic is related to the competency characteristic through a successive chain of dependencies across some characteristics, we can start to formulate a plausible generation process for an applicant distribution.

The joint distribution defined by a bayesian network, with characteristics as its random variables, meets the three requirements:

- If we accept that the correlation between the protected characteristic and competence arises from a system of causal influences, as most explanations for bias suggest, then the real-world distribution can be modelled by some bayesian network.
- Random generation of bayesian networks is difficult but feasible.
- Representing the applicant distribution as a system of causal dependencies mirrors our own intuitions of causal relationships between unknowns.

Therefore, the applicant distribution is the joint distribution given by some randomly generated bayesian network $\mathcal{B} = (G, \Phi)$. G is a graph over the nodes $V = \mathbf{C} = \{C_0, C_1, \dots, C_n\}$ and we have:

$$\mathcal{D} = P_{\mathcal{B}}(C_0, \dots, C_n) = \prod_{i=0}^n P(C_i \mid \text{Parent}_G(i))$$

I assume that the protected characteristic isn't affected by some other characteristic (often it denotes a feature an applicant cannot change^[45]) and thus that the corresponding node has no parents. Additionally, I assume competence is not a direct or proximal cause of the protected characteristic. Therefore, I constrain the generated network such that the protected characteristic is neither a direct parent or a grandparent of competence. Together:

$$\text{Parent}_G(Z) = \emptyset \quad Z \notin \text{Parent}_G^{(\leq 2)}(Y)$$

I discuss random bayesian network generation in Section 3.1.



Figure 2.1: An example of what the nodes in a generated bayesian network may model in reality.

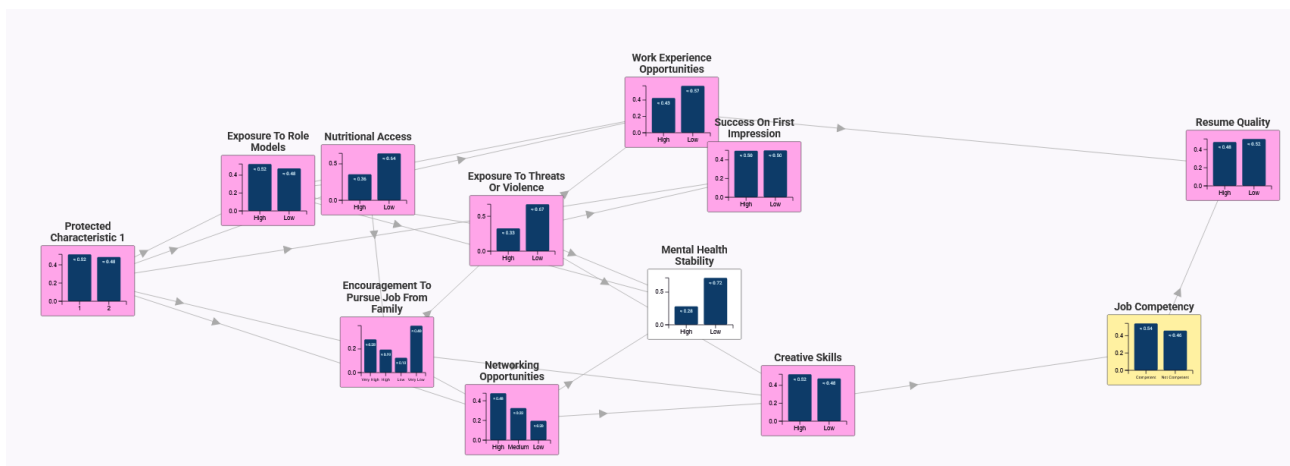


Figure 2.2: Another example.

A Run of the Simulation

Each simulation run proceeds as follows:

1. Generate a random bayesian network and derive from it an applicant distribution.
2. Sample applicants for training, for testing, and for calibrating post-training mitigations.
3. Generate an application from each applicant.
4. Train a recruiter on the training set of applications and associated competences.
5. Apply any post-training bias mitigations to the recruiter.
6. Use the recruiter to classify each application in the training set.
7. Measure the bias exhibited in the decisions made by the recruiter.

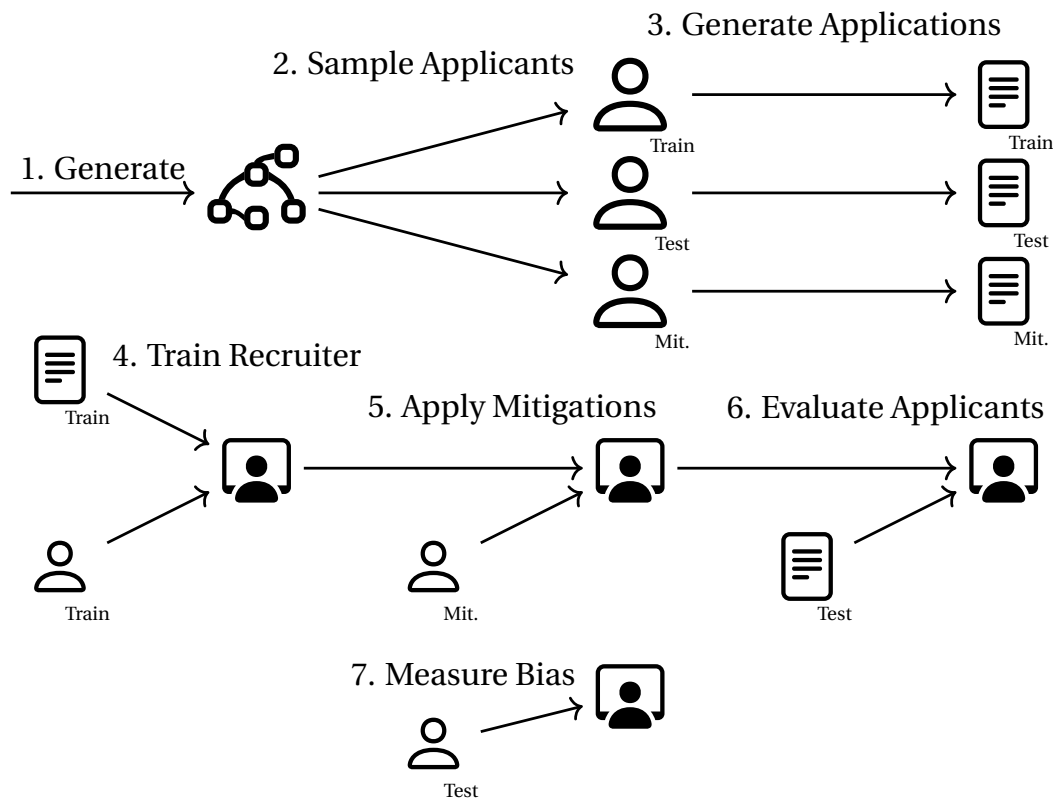


Figure 2.3: A run of the simulation

I repeat this simulation with three variations.

First, I vary the recruiter R , experimenting with different modern ML techniques. I hypothesise that recruiters which can better abstractly represent the actual causal relationships between variables are less biased, and this experiment will provide evidence for or against this idea. I describe the recruiters I implement in Section 3.3.

Next, I vary $\mathcal{A}$ to modify which characteristics appear in the applications. A standard recruitment practice is to hide protected features from recruiters, and this allows us to assess the practice empirically. I discuss application generation in Section 3.2.

Finally, I change the recruiter's default decision function d to try and mitigate bias after training. Specifically, we can adjust thresholds at the protected-group level. I discuss post-training bias mitigations in Section 3.5.

Together, the modelling decisions I've made include a large number of simplifying assumptions. I justify them and discuss their limitations in Section 4.2.

2.4 Tools Used

The relevant license was verified before each tool was used. These licenses are cited immediately after each tool's name.

Simulation

Tool	Use in Project	Justification	Starting Point
Python ^[46]	• Simulation • Website backend • Data analysis	• The dominant language for data science • Large ecosystem of data science libraries • Scripting language, so it allows fast iterative development • Its slow speed is mitigated by libraries implemented in C/CUDA	Experienced. This is my main coding language
pgmpy ^{[47][48]}	• Storing, conditioning, sampling from BNs • Using HC search to approximate a BN for the bayesian recruiter	• Standard Python library for categorical BNs • Means I don't have to reimplement BN functionality	No experience
(PyMC) ^{[47][4]}	• Testing continuous BNs	• Heavier library for sampling from and storing continuous BNs • Best library for continuous BNs so useful in requirements refinement • Can only approximate categorical distributions and sampling from them is much slower so this isn't used in the final codebase	No experience
networkx ^[50]	• Storing and manipulating graphs	• Standard graphing library for Python • Used by pgmpy	Minimal experience
scikit-learn ^[51]	• Implementations for four of the recruiters	• Provides out-of-box solutions for four recruiters	Minimal experience
PyTorch ^[52]	• Implementations for the encoder and MLP recruiters	• Industry-standard for deep learning • I have experience from DNN module this year	Minimal experience
Pandas ^[53]	• Data Analysis	• Industry-standard for analysing data • I have experience from ADS module this year	Some experience. One self-contained data science project
Matplotlib ^[54] Seaborn ^[55]	• Plotting and visualisation	• Customisable tools for Python data-plotting	Experienced
scipy ^[56]	• Dual-annealing for mitigations	• Provides out-of-box solution for one difficult optimisation problem	Minimal experience

Visualisation and Walkthrough

Tool	Use in Project	Justification	Starting Point
Typescript ^[5] HTML ^[58] Svelte ^[59]	• Website frontend	• I have familiarity with this stack from the group project • Additional familiarity with typescript from summer internship	Experienced
Tailwind ^[60] Skeleton.dev ^[61]	• Design system	• Design not a major part of the project, so no need to reimplement standard functionality	Experienced
D3.js ^[62]	• Network visualisation	• Provides out-of-box force simulations and graph rendering • Easily customisable and interacts well with Svelte	No experience
Django ^[63]	• Website backend	• Built in Python so interacts with simulation code	No experience
Gsap ^[64]	• Animations	• Standard library for animating HTML • Solution found online for type-safe interacting with Svelte	No experience

Other

Tool	Use in Project	Justification	Starting Point
GitHub ^[65] Git ^[66]	• Version control	• Meant work was backed up • Allowed me to host and track progress online	Experienced
Google Cloud ^[67]	• Compute for running simulation	• 3-month free trial is appealing, even though I have more experience in AWS	Minimal experience
Supabase ^[68]	• Database for storing simulation runs	• Meant multiple computers could run simulations at once and results are stored in a centralised place. • Free subscription tier • Requires minimal setup	No experience
PyCharm ^[69]	• Development IDE	• QOL features: fast searching, saving run configurations, code formatting • Pythonic	Experienced
Ionos ^[70]	• VPS for web hosting	• Cheap and easy to set up • Discount for first year of domain	No experience
Nginx ^[71]	• Reverse proxy for VPS	• Easy to setup	No experience

2.5 Proposal Refinement and Requirements

I present a refined set of requirements:

1. Implement generating a random categorical bayesian network $\mathcal{B} = (G, \Phi)$ with $V = (C_0, \dots, C_n)$ and deriving from it an applicant distribution $\mathcal{D} = P_{\mathcal{B}}(C_0, \dots, C_n)$.
2. Implement generating an application index set $\mathcal{A} \subseteq \{1, \dots, n\} \setminus \{y\}$.
3. Implement generating samples $\mathbf{c} = (\mathbf{c}_0, \dots, \mathbf{c}_n)$ from an applicant distribution $\mathcal{D}$, and deriving an application $\mathbf{x} = (\mathbf{c}_i : i \in \mathcal{A})$ from each sample.
4. Implement recruiters using modern ML techniques to classify applications, including a logistic regression, a neural network, and a recruiter which constructs a bayesian network.
5. Implement bias measurements based on demographic parity, equalised odds and predictive parity.
6. Implement post-training bias mitigations, including one which adjusts thresholds to be the same at the protected-group level.
7. Run the simulation in Section 2.3 and record and analyse results.
8. Build and host a visualisation of bayesian networks, the generation process, and the bias measurement process.
9. Investigate whether a recruiter's architecture is predictive of bias.
10. Investigate whether the application generation condition is predictive of bias.
11. Investigate the consistency of different bias metrics.
12. Investigate the usefulness of post-training bias mitigations.
13. Build and host a walkthrough of the research.

This project was an exploratory one, and proposal refinement continued into the implementation stage.

The initial requirements are given in the project proposal (Section B). At their creation, I was less familiar with fairness in machine learning and under the impression that existing metrics for bias exhibited agreement. I was surprised by my initial results, which contradicted that assumption and suggested that standard explanations for bias were incomplete. I decided that my results would be valuable to a non-technical audience, but that they required the context of the research to understand. This motivated the creation of an online visualisation and walkthrough.^[72]

At the time of project proposal, I was lacking experience in practical data science and machine learning (I took the Advanced Data Science^[73] and Deep Neural Networks^[74] modules this year). Improving my skills gave me the flexibility to expand the scope of the project, which adapted as I gathered preliminary results.

The final change to the requirements was motivated by the insight that any conclusions I could draw from experiments on real resumes would be limited by these datasets themselves being biased approximations of the ground truth. Additionally, structured applicant datasets were difficult to find. For that reason, I deprioritised experiments on real datasets.

2.6 Software Methodology

Due to the project's exploratory nature, I used an approach inspired by agile methodologies, focusing on quickly writing code which could be easily adapted, and then performing large refactors once the requirements became clear. The code was modularised and became object-oriented during these refactors, so that I could abstract and reuse implementations once they were unlikely to undergo further changes. The simulation was difficult to conceptualise, so I used types within my backend and frontend code.

I used GitHub^[75] for version control and took notes on paper and digitally. I also maintained a Kanban^[76] to track progress.

2.7 Ethical Considerations

The project could have clear positive ethical implications if I uncover unfair working practices or it provides insight into bias. However, there are some risks. If a casual reader believes I imply that there is a causal relationship between membership of some protected group and competence, or indeed any other characteristic, then that could have consequences for that protected group. For that reason, I generally avoid any reference to particular groups or explicit labelling of characteristics, and make it clear throughout the project that this is simulated data.

I tried to have informal discussions with people with different perspectives, particularly in regard to the potential sensitivity of opinions expressed. I am also aware that this research is currently topical. For example, DEI is under criticism by the Trump administration,^[77] and the UK Supreme Court's ruling on the interpretation of women in the Equality Act is likely to have some implications for hiring practices.^[78] I avoid further reference to these subjects as I don't have the political or sociological background to discuss them adequately, but I remain aware of them in communication.

3 Implementation

Following from the design in Section 2.3, I provide additional details on each stage of a simulation run. After some experimentation, I decide to generate 80 nodes for each bayesian network, and sample 10,000 applicants for each simulation run. That is, 50% for training, 25% for mitigation, 25% for calibrating post-training mitigations.

3.1 Generating a Bayesian Network

To make precise what is meant by a random bayesian network, I motivate what would constitute a valid network. First, if the network were too dense, it would reduce the complex dependencies it could model. If more variables are related, then what would have been conditional dependencies are instead modelled as direct dependencies. Therefore, I constrain each node except two to have between 2 and 3 parents:

$$\forall v \in V / S. \ 2 \leq |\text{Parents}(v)| \leq 3 \quad |S| = 2$$

Second, if the network had low mutual information^[79] between the random variables corresponding to neighbour nodes, a recruiter would be unable to predict the applicant's competence with high accuracy. If a recruiter's accuracy were low in real-world recruiting, it should not be relied upon for decision-making. Therefore, I also constrain that the mutual information between each characteristic and its parents is between 50% and 90% of its theoretical maximum:

$$\forall v \in V. \ 0.5 \leq \frac{I(v, \text{Parents}(v))}{I_{\max}(v, \text{Parents}(v))} \leq 0.9$$

Finally, recall that our design in Section 2.3 constrains a valid bayesian network to be weakly connected, requires the protected characteristic to be an orphan and requires the protected characteristic to be neither a parent nor grandparent of the competence characteristic:

$$\forall v_i, v_j \in V. \ v_i \leftrightarrow \dots \leftrightarrow v_j \quad \text{Parent}_G(Z) = \emptyset \quad Z \notin \text{Parent}_G^{(\leq 2)}(Y)$$

Generating a random Directed Acyclic Graph

The `networkx` package provides multiple functions to generate a random DAG,^[80] but I disregarded these because the generation processes are not uniformly random. To avoid any bias from patterns in the structure of the network, each valid DAG should have the same chance of being generated.

Building a uniform DAG generation process is challenging, and two strategies exist to do so:^[81]

- Create an irreducible, reversible and aperiodic Markov chain of DAGs. The stationary distribution of that Markov chain is uniform, so one can sample a uniform DAG by running it until it reaches adequate convergence. This method requires $O(n^4)$ samples to be drawn from the Markov chain, and I reject it for poor performance.
- Build an enumeration of all directed acyclic graphs and sample uniformly from that. An implementation of this strategy is given by Kuipers and Moffa,^[81] and I implement it with modification.

An outpoint of some DAG G is a node with no parents.^[82] Note that by removing G 's outpoints and the connecting edges, we produce a new smaller DAG G' with its own set of outpoints:

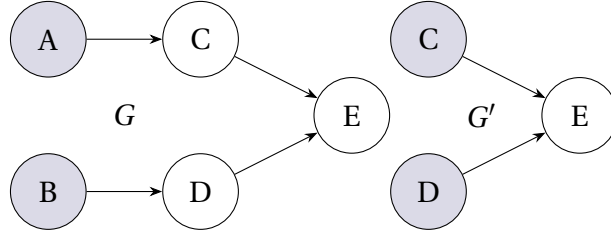


Figure 3.1: G and G' . Shaded nodes are outpoints.

Let $a_{n,k}$ be the number of valid DAGs with n nodes and k outpoints. Given a table of all $a_{n,k}$, an algorithm can be given to uniformly generating a valid DAG with n nodes:

1. Generate a value k , where k is how many outpoints should be in the generated DAG:
 - (a) Generate a random number $r \in_R (0, \sum_{i=1}^n a_{n,i} + a_{n,2} + \dots a_{n,n})$.
 - (b) Choose the smallest k such that $r < \sum_{j=1}^k a_{n,j}$.
2. Recursively generate a random DAG G' with $n - k$ nodes.
3. Connect the k outpoints in G to nodes in G' at random. For each outpoint:
 - (a) Generate the number of parents p in G' which that outpoint should have. This is generated from the binomial distribution $\text{Bin}(n - k, 0.5)$ truncated to be in the range $(2, 3)$, then normalised.
 - (b) Randomly sample p nodes in G' and create an edge from each sampled node to the outpoint.
 - (c) If the resulting DAG has more than k outpoints, reject it and repeat this step.

In order to generate $a_{n,k}$, I derive the recurrence relation:

$$a_{n,k} = \sum_{s=1}^m a_{m,s} \cdot c_{m,s,k}$$

where $m = n - k$

- $a_{n,k}$ is the number of valid DAGs with n nodes and k outpoints.
- $c_{m,s,k}$ is the number of DAGs which can be built by adding k outpoints to a fixed DAG with m nodes and s outpoints.
- Then $a_{m,s} \cdot c_{m,s,k}$ is the number of DAGs which can be built by adding k outpoints to *any* DAG with m nodes and s outpoints.
- So I calculate $a_{n,k}$ by summing the counts of valid DAGs for each possible number of outpoints.

$c_{m,s,k}$ can be calculated without computational challenge, so I use a two-dimensional dynamic programming approach to calculate $a_{n,k}$ by producing the entire matrix a , using the previous entries in the matrix to compute each successive entry. Deriving c is mathematically complex, involving the inclusion-exclusion principle, so I leave it to the appendix (Appendix A.2).

Assigning Conditional Probability Distributions

I build a valid bayesian network $\mathcal{B} = (G, \Phi)$ from a DAG G by assigning a CPD Φ_C for each node, with the constraint that the mutual information between each characteristic and its parents is between 50% and 90% of its theoretical maximum. I was unable to create a uniform random generation process for CPDs, and instead, we use a greedy approach to approximate one.

A Dirichlet distribution $\text{Dir}(\alpha)$ is the standard conjugate prior for categorical distributions.^[83] $|\alpha|$ is negatively correlated with the distribution's entropy.^[84] I generate a random conditional probability distribution $P(A = a \mid W = w)$ by generating $P(A = a) \sim \text{Dir}(\alpha)$ for each instantiation w of W .

For each node in the graph then, I use this process to generate a random conditional probability distribution. Until the mutual information condition is satisfied, I reject the distribution and regenerate it.

Under some conditions, this algorithm gets stuck. That is, there is no conditional probability distribution which satisfies the mutual information requirement. In this case, I rerun the entire cpd-assigning process with a lower α . Initially, $\alpha = 1$ and I halve α at each unsuccessful generation.

3.2 Generating Applicants and Applications

Given a bayesian network, I sample an applicant from the joint distribution it defines by sampling each characteristic with its associated conditional probability distribution, in the topological order denoted by the DAG. The pgmpy package implements this.^[85]

Recall that an application is a subvector $\mathbf{x} = (\mathbf{c}_i : i \in \mathcal{A})$ of an applicant, where $\mathcal{A} \subseteq \{1, \dots, n\} \setminus \{y\}$ is a fixed index set which excludes the competence characteristic. In our design of the simulation, I omitted details on how I randomly generate $\mathcal{A}$ for each simulation run. I derive four different generation processes for $\mathcal{A}$. There is a visualisation of each on the website.^[86]

A **Proxy Characteristic** C_p is a characteristic connected to the protected characteristic in the bayesian network which defines the applicant distribution:

$$C_p \in \text{Neighbours}_{\mathcal{B}}(C_z)$$

A **Distant Proxy Characteristic** C_{dp} is connected to the protected characteristic by one or two edges:

$$C_{dp} \in \text{Neighbours}_{\mathcal{B}}^{(2)}(C_z) \vee C_{dp} \in \text{Neighbours}_{\mathcal{B}}(C_z)$$

Each applicant reveals the same set of 10 characteristics drawn randomly from all non-competency characteristics:

$$\mathcal{A} \sim \text{Uniform}(\{A \subseteq \{1, n\} \setminus \{y\} : |A| = 10\})$$

For each condition, I specify an additional requirement for a valid $\mathcal{A}$:

1. **Include Protected** - $\mathcal{A}$ includes the protected characteristic: $z \in \mathcal{A}$.
2. **Omit Protected** - $\mathcal{A}$ does not include the protected characteristic: $z \notin \mathcal{A}$
3. **Include Proxies** - $\mathcal{A}$ includes exactly five proxy characteristics:

$$\mathcal{A} = \{p_1, p_2, p_3, p_4, p_5, c_1, c_2, c_3, c_4, c_5\} \wedge \forall i. p_i \in \text{Neighbours}_{\mathcal{B}}(z) \wedge \forall i. c_i \notin \text{Neighbours}_{\mathcal{B}}(z)$$

4. **Omit Distant Proxies** - $\mathcal{A}$ includes no distant proxy characteristics:

$$\forall c \in \mathcal{A}. C_c \notin \text{Neighbours}_{\mathcal{B}}^{(2)}(C_z) \wedge C_c \notin \text{Neighbours}_{\mathcal{B}}(C_z)$$

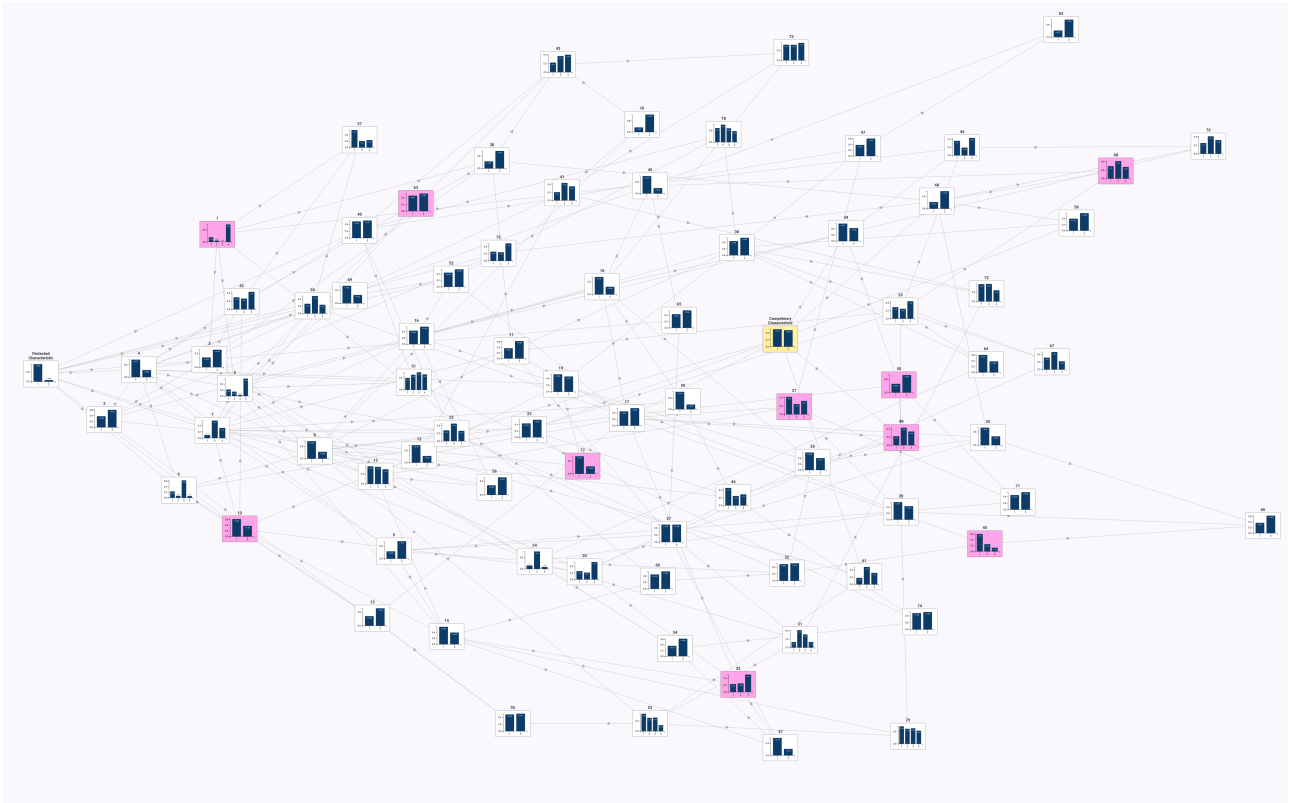


Figure 3.2: A valid bayesian network generated with the Omit Protected condition. Pink nodes are in the application and the yellow node is the score node.

3.3 Recruiters

Recall that a recruiter is an ML model which trains on a set of applications and corresponding labels in {competent, not competent} and then classifies each new application into {hire, reject}. I implement seven recruiters, chosen to be a performant cross-section of modern ML techniques. Exact details are left to the appendix, but parameters were chosen to be realistic, without causing significant detriment to performance:

Recruiter	Implementation
Random Forest, Support Vector Machine, Logistic Regression	scikit-learn
Bayesian	pgmpy
Deep/Shallow Multi-Layer Perceptrons, Encoder-Only Transformer	PyTorch

The first three of these were implemented straightforwardly with scikit-learn.^{[87][88][89]}

The bayesian recruiter generates a bayesian network $\mathcal{B} = (G, \Phi)$, where the input features and target feature are the random variables. The structure of the DAG G and the conditional probability distributions in Φ should maximise the likelihood of the training data.

To estimate the structure of G , the bayesian recruiter uses hill climb search, an optimisation approach which iteratively changes G to maximise a score function. The score function favours edges with higher mutual information and penalises DAG complexity. After the optimal G is constructed, the recruiter uses maximum likelihood estimation to estimate Φ . This is implemented with pgmpy.^{[90][91]}

I provide two implementations of neural networks, both of which are implemented using alternating Linear and ReLU layers from the PyTorch package.^{[92][93]} The deep network is composed of 16 linear layers of 16 perceptrons each, and the shallow network is composed of 2 linear layers of 64 perceptrons each. These neural networks were then trained with PyTorch's Adam optimiser,^[94] with sensible parametrisation.

The transformer recruiter models the modern transformer architecture, used in state-of-the-art generative models. I implement a standard encoder-only transformer with 2 attentional heads, and 4 layers of 16 neurons each.^[95] The network was trained with the Adam optimiser.

3.4 Measuring Bias

I specify three criteria for a measurement of bias:

1. It should be zero if the recruiter is unbiased.
2. It should diverge from zero monotonically as the recruiter's bias does.
3. It should correspond to some intuitive real-world property.

For the experiments, I limit the protected characteristic to be a binary variable, so each applicant is from group A or group B. This allows us to denote the direction of bias as the sign of the measurement: If the measurement is positive, then bias disadvantages group A. If the measurement is negative, then bias disadvantages group B. All the measurements below are intuitive to an extent. I provide a way to conceptualise each one in Appendix A.5.

Demographic parity is satisfied if the hiring rate is equal across groups. A metric which satisfies our three criteria is the difference between hiring rates:

$$\text{Hiring Rate Difference} = \text{Hiring Rate}_A - \text{Hiring Rate}_B$$

Equalised odds is satisfied if the false negative rate and false positive rate are equal across groups. From this, I derive two metrics:

$$\text{FNR Difference} = \text{False Negative Rate}_B - \text{False Negative Rate}_A$$

$$\text{FPR Difference} = \text{False Positive Rate}_A - \text{False Positive Rate}_B$$

Note I have flipped the signs for FNR Difference because a higher false negative rate is a disadvantage for a group. Equalised odds is satisfied if $\text{FNR Difference} = 0 \wedge \text{FPR Difference} = 0$.

Predictive Parity is satisfied if the false discovery rate and false omission rate are equal across groups. From this, I derive two metrics:

$$\text{FDR Difference} = \text{False Discovery Rate}_A - \text{False Discovery Rate}_B$$

$$\text{FOR Difference} = \text{False Omission Rate}_B - \text{False Omission Rate}_A$$

I have flipped the signs for FOR Difference because a higher false omission rate is a disadvantage for a group. Predictive parity is satisfied if $\text{FDR Difference} = 0 \wedge \text{FOR Difference} = 0$.

3.5 Applying Post-Training Bias Mitigations

Recall I don't model a recruiter as returning a binary decision from each application, but rather a score in the interval (0, 1). The recruiter's default decision rule is:

$$\hat{y} = \begin{cases} \text{Hire} & \text{if } h_\theta(\mathbf{x}) \geq h \\ \text{Reject} & \text{if } h_\theta(\mathbf{x}) < h \end{cases}$$

where h is a threshold chosen such that the proportion of competent candidates in the training set is the same as the proportion of hired candidates in the testing set.

It would be natural to adjust those thresholds at the group level. For example, I satisfy demographic parity by adjusting the thresholds such that for each group, the proportion of hired applicants in the training set is equal to the proportion of competent applicants in the testing set.

Such a post-training mitigation strategy is attractive because it is a cost-effective and universal solution to any real-world classification problem which demands demographic parity to be satisfied. If it were effective for all definitions of machine fairness, it would mean that bias doesn't need to be considered throughout the development process, and existing recruiters can be modified to satisfy the condition without retraining. However, adapting this mitigation strategy to satisfy equalised odds or predictive parity is more difficult because there is no obvious analytical method for how to choose these thresholds.

For a binary protected characteristic, our decision rule is modified to the following:

$$\hat{y} = \begin{cases} \text{Hire} & \text{if } h_{\theta}(\mathbf{x}) \geq h_a \wedge \mathbf{c}_z = a \\ \text{Hire} & \text{if } h_{\theta}(\mathbf{x}) \geq h_b \wedge \mathbf{c}_z = b \\ \text{Reject} & \text{otherwise} \end{cases}$$

Where h_a and h_b are thresholds of our choosing. I constrain that the hiring rate in the training set is the same as the competence rate in the testing set, so there is a single degree of freedom in these thresholds.

As well as the training set and testing set, I sample a separate set of **Mitigation Applicants**. After training each recruiter, I measure the bias it exhibits in its decisions, given some h_a and h_b . I minimise bias by varying h_a and h_b , with the additional constraint that the hiring rate and competence rate in the training set must be the same. This is a constrained optimisation problem which I solve with dual annealing, implemented with the SciPy package.^[96]

In practice, I find that no value of h_a and h_b can satisfy equalised odds or predictive parity. In the mitigation set, we can satisfy only one of equality in false negative rates and false positive rates, or only one of equality in false discovery rates and false omission rates. Even then, this often doesn't generalise to the testing set.

I experimented with ten different post-training bias mitigations, which I summarise below. Note that all mitigations ensure that the hiring rate in the testing set is the same as the competence rate in the training set:

Mitigation	Description
No Mitigation	Equal group thresholds
Satisfy Demographic Parity	Hiring rate in each group is the overall competence rate
Satisfy Proportional Parity	Within each group, hiring rate is competence rate
Optimise for FNR Equality	Minimise FNR Difference ²
Optimise for FPR Equality	Minimise FPR Difference ²
Optimise for Equalised Odds	Minimise FNR Difference ² + FPR Difference ²
Opt. for Equalised Odds and Accuracy	Minimise FNR Difference ² + FPR Difference ² + (1 – Accuracy) ²
Optimise for FDR Equality	Minimise FDR Difference ²
Optimise for FOR Equality	Minimise FOR Difference ²
Optimise for Predictive Parity	Minimise FDR Difference ² + FOR Difference ²
Opt. for Predictive Parity and Accuracy	Minimise FDR Difference ² + FOR Difference ² + (1 – Accuracy) ²

3.6 Experimenting with the Simulation

I ran two different experiments with the simulation.

Firstly, for each of the four application generation conditions (listed in Section 3.2) and seven unmitigated recruiters (listed in Section 3.3), I ran the simulation across ≈ 1200 simulation runs each. From this, I concluded that the random forest recruiter was the most accurate of all recruiters, and also the recruiter which was least biased by all five metrics.

Second, I ran the simulation with only the random forest recruiter and all 11 mitigations (listed in Section 3.5) across ≈ 4000 simulation runs. Post-training bias mitigations were not performant, and I wanted to collect the majority of my data on unmitigated recruiters, which is why I didn't perform a single larger experiment.

Overall, this accounted for about six days of computation time. I was aware of this, so I used a virtual machine from Google Cloud to run simulations and a remote database from Supabase to store their results.^{[97][98]} This was challenging. I had no experience with the process and it necessitated replicating my environment in Linux.

After collecting data, I rigorously analysed it within a jupyter notebook.^[99] I used a standard data analysis workflow: first pre-processing data, then exploring general patterns and trends, and finally answering specific questions and validating hypotheses. After an exhaustive analysis, I decided that my most important findings could be summarised. For each of these, I critically tested and justified the finding. I list my findings in Section 4.1, and provide some example code for data analysis in Appendix A.6.

3.7 Visualisation and Walkthrough

Implementing the visualisation^[100] and walkthrough^[101] was challenging. Due to space constraints, I only give an overview of the issues I encountered.

I chose to use a Typescript frontend and a Python backend, communicating between them with REST requests^[102] and websockets.^[103] It would have been inefficient to send each user's active bayesian network with each request, so to avoid this, I implemented user-level caching and tracking of session data. I bought and registered a domain name, and I use a VPS with a reverse proxy as the website server.

For the visualisation,^[100] I concisely represent a non-planar graph as well as all the details of the comprising characteristics. I drew inspiration from Bayes Server^[104] for the conceptual design, and developed the visualisation with D3.js,^[105] a low-level plotting library.

3.8 Website Demonstration

I provide some screen captures of the website at modelling-bias.com:

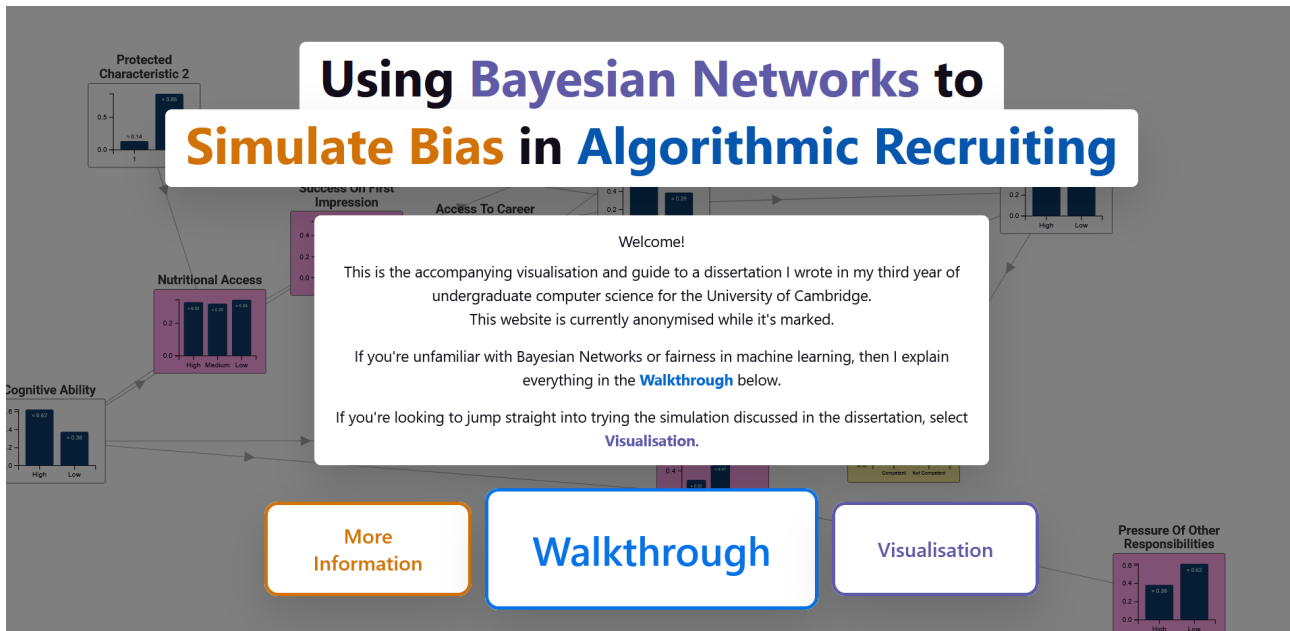


Figure 3.3: The website's landing page.

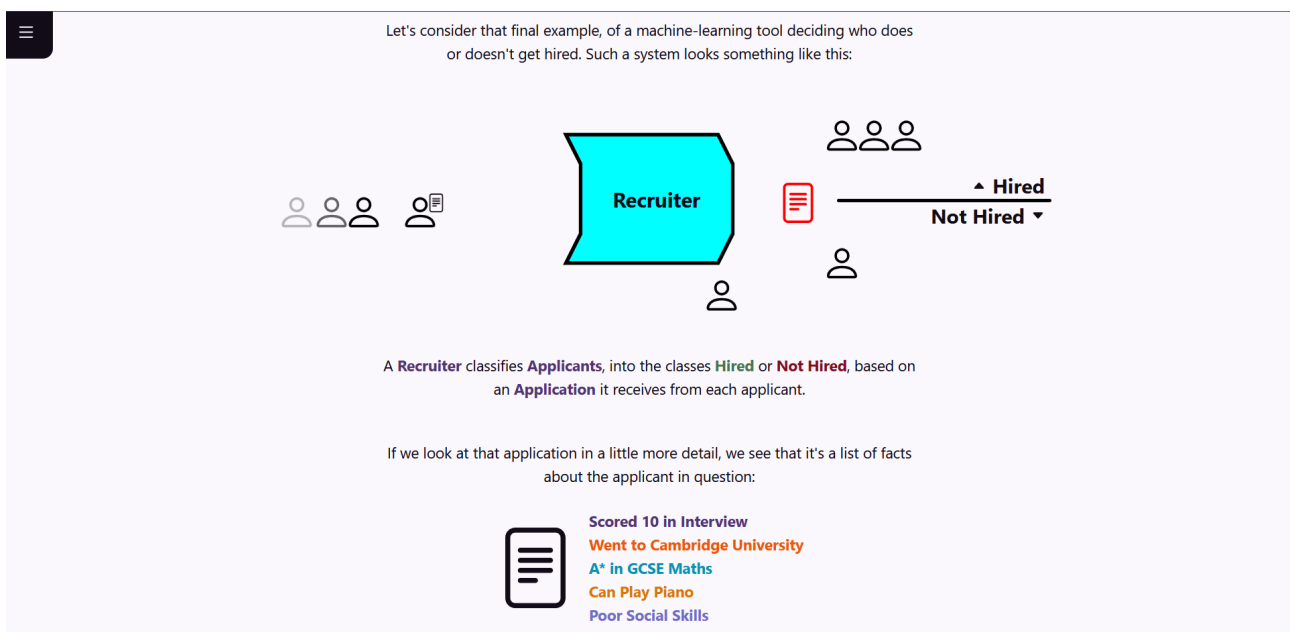


Figure 3.4: An extract from the walkthrough. The first diagram is an animation.

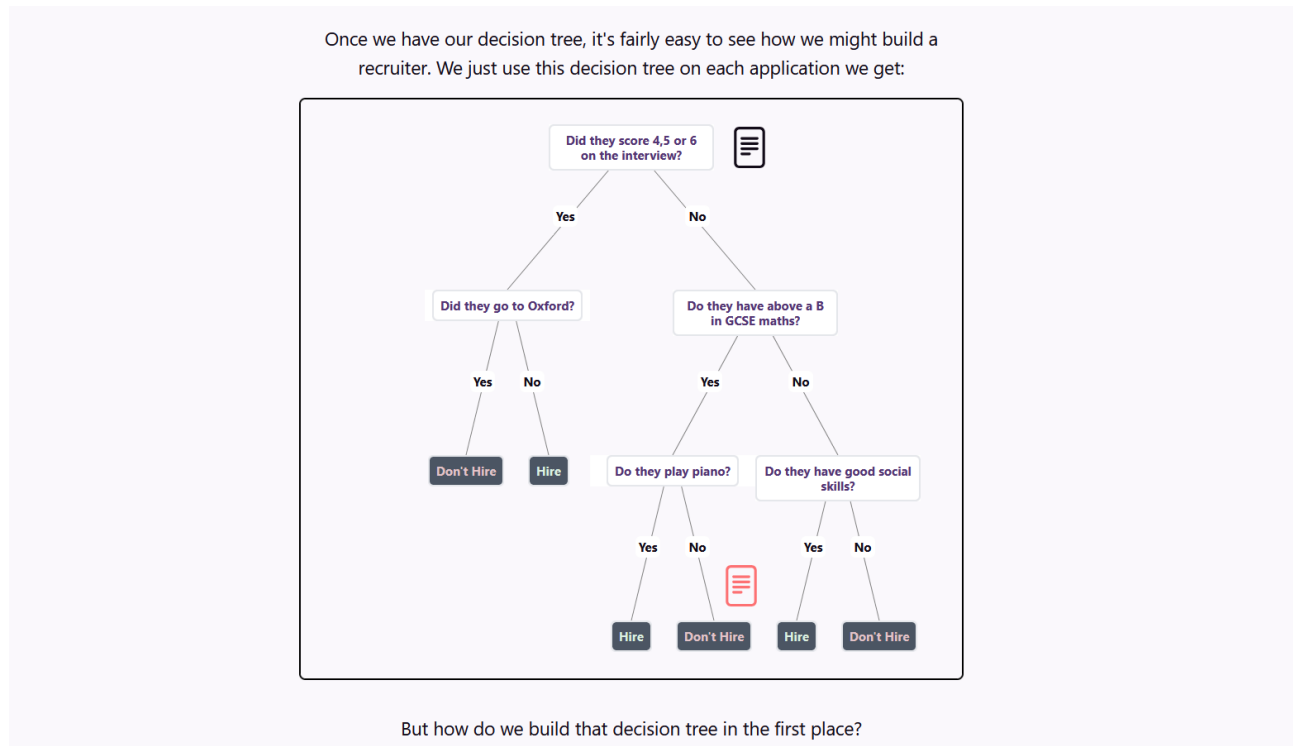


Figure 3.5: Another extract from the walkthrough, explaining the random forest recruiter with an animation.



Figure 3.6: Using the visualisation to explore how conditioning a variable affects a bayesian network.

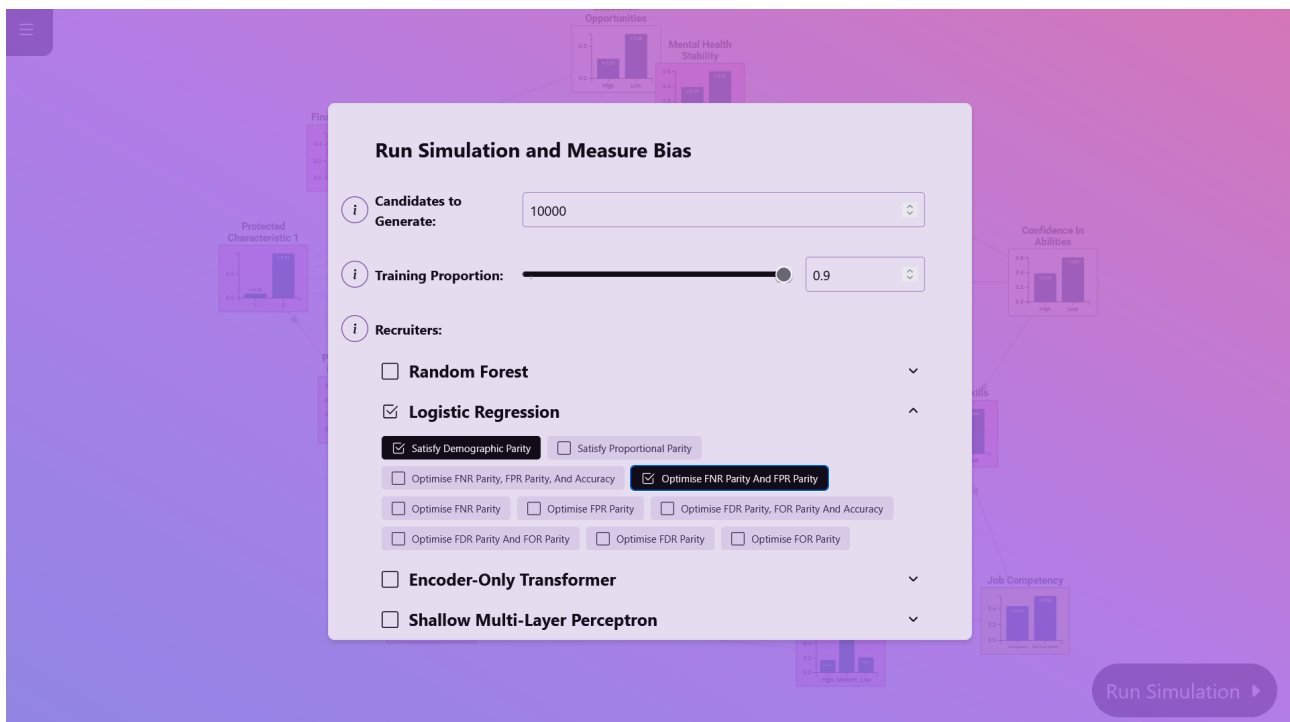


Figure 3.7: The interface for running a simulation.

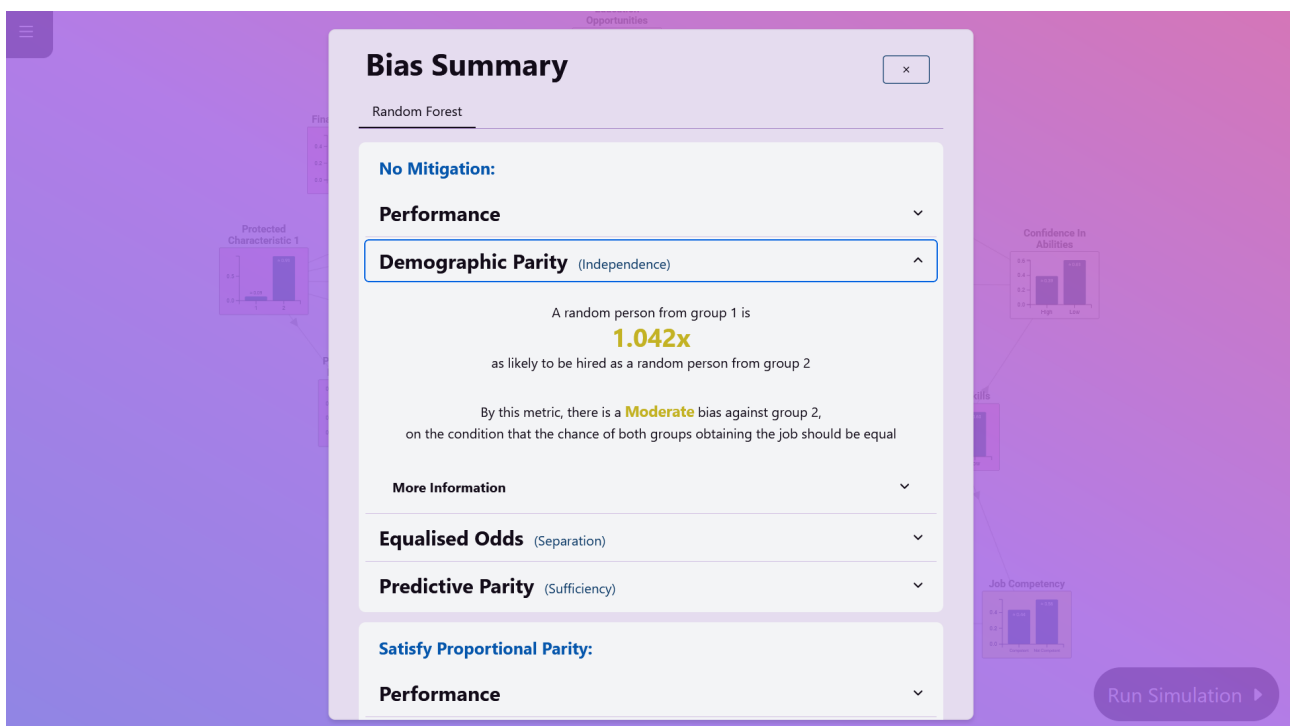


Figure 3.8: The interface for summarising how much bias each recruiter is measured to exhibit.

3.9 Repository Overview

All code produced is my own except configuration files, the libraries listed in the tools section, and some credited instances of general utility functions adapted from online solutions. This is a non-exhaustive overview.

The Backend



Much of the project is object-oriented, using the language I developed in Section 2.3:

- `characteristic.py` implements a characteristic with associated metadata.
- `bayesian_network.py` abstracts a bayesian network of characteristics, with functionality for sampling, conditioning and renaming variables. I provide two implementations:
 - `pgmpy_network.py` uses pgmpy to implement categorical bayesian networks.
 - `pymc_network.py` uses PyMC to implement continuous bayesian networks. This became redundant when the scope of the project was constrained to explore categorical bayesian networks.
- `recruiter.py` abstracts a recruiter which makes decisions on a set of applicants. I implement seven recruiters in files of the form `*_recruiter.py`.
- `applicants.py` implements a population of applicants as a set of samples from a bayesian

network. Each applicant can generate applications.

- [mitigation.py](#) abstracts a post-training bias mitigation. I provide eleven implementations of this, which account for the rest of the files in the [categorical_bias_mitigation](#) folder.
- [recruiter_bias_analysis.py](#), [mitigation_bias_analysis.py](#) and [group_prediction_information.py](#) store and measure of bias for each simulation run.

[main.py](#) is the entry point of the program:

1. It generates a random bayesian network with [generate_categorical_network.py](#)
2. It samples a random set of applicants from the network with [sample_applicants.py](#)
3. It instances recruiters from [recruiters](#) and mitigations to be used from [categorical_bias_mitigation](#)
4. It runs the simulation on those applicants and recruiters with [simulate.py](#)

[simulate.py](#) takes a set of applicants and recruiters and returns recruiter bias measurements:

1. It splits the applicants into a test, training and mitigation set.
2. Each applicant generates an application.
3. For each recruiter:
 - (a) The recruiter is trained on the testing set.
 - (b) The mitigation is initialised with the mitigation set.
 - (c) Bias is measured.

[generate_categorical_network.py](#) returns a random bayesian network:

1. It generates a random DAG with [generate_dag.py](#)
2. It selects which characteristics should be included in the application, which is the protected characteristic and which is the competency characteristic with [choose_characteristics.py](#).
3. It assigns random conditional probability distributions to satisfy a mutual information requirement with [assign_cpds.py](#).

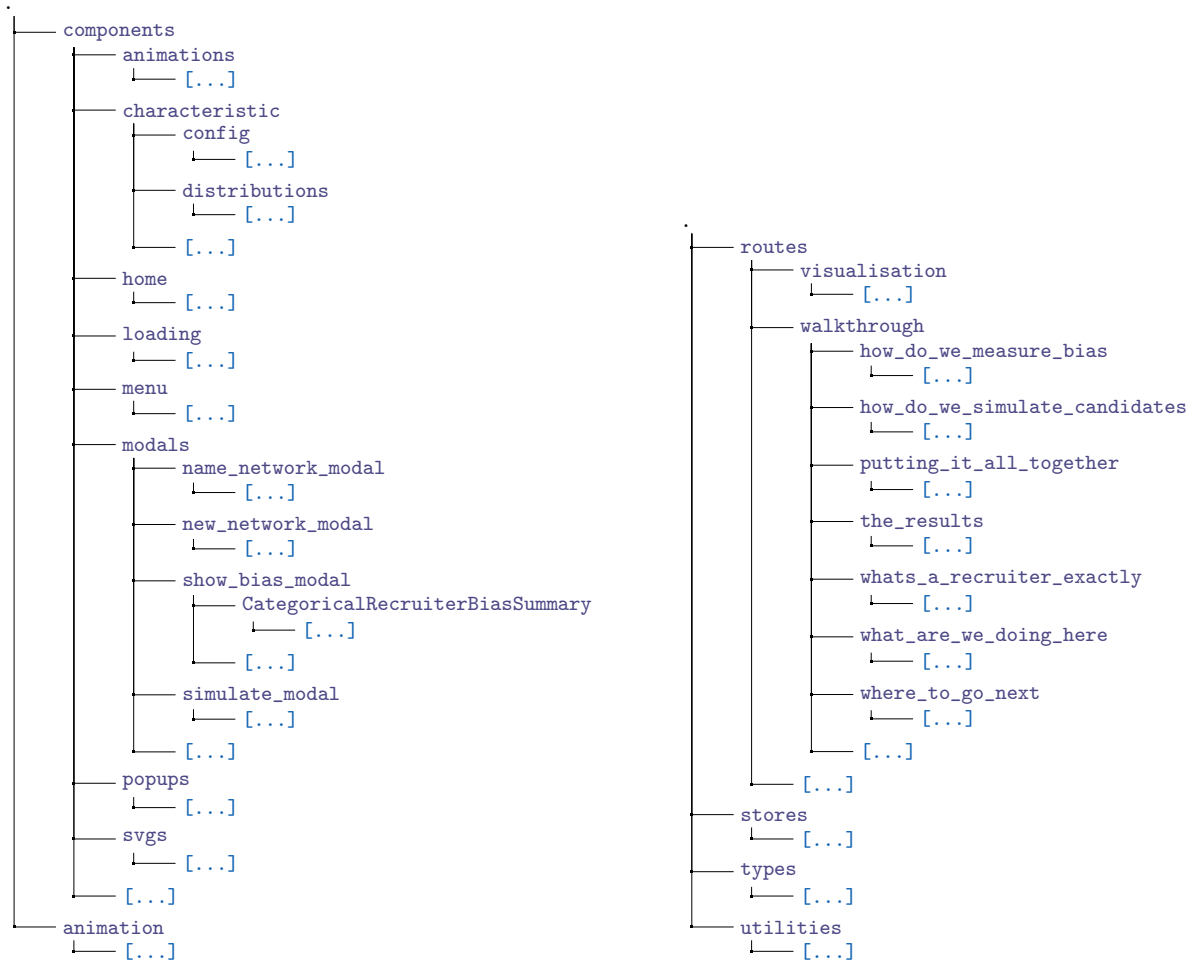
[experiments](#) contains implementations for each experiment I run. Each experiment is of the form:

1. Setup the experiment with [setup_experiment.py](#)
2. Run the simulation with [simulate.py](#)
3. Save the results to a database with [save_to_db.py](#)

This code also implements the backend for the website [modelling-bias.com](#), and [api](#) handles communication with the frontend:

- The types in [request_types](#) and [response_types](#) set out the form of information expected in the frontend.
- Each consumer of the form [*_consumer.py](#) manages a WebSocket, which enables two-way communication with the frontend, for example to report progress on simulating.
- Endpoints which don't require a WebSocket are specified in [urls.py](#) and implemented in [views.py](#).
- [cache.py](#) provides implementations for using the cache to store necessary data on each user, so the full network doesn't have to be sent around each time. Django implements this as an SQLite database in [db.sqlite3](#).

The Frontend



The **components** folder contains reusable frontend components, which is the unit of modularity native to Svelte. Within this folder, I organise based on functionality

- The **animations** folder contains each animation (in component form) used in the walkthrough.
- The **characteristic** folder contains the components needed to implement a single characteristic display on the network visualisation graph. The **config** folder implements the drop-down buttons when each characteristic is expanded, and the **distributions** folder implements a component to display the categorical probability distribution.
- **home** contains various components implemented on the page landing screen, **loading** contains various components which implement loading displays while the backend is waited on.
- **modals** implements four different modals: full page menus which appear in front of page content. These are provided by the Skeleton.dev design system and customised here.

The **routes** folder denotes the navigational layout of the website in the format given by Svelte. The website consists of a non-technical walkthrough, and a visualisation of the simulation and bayesian networks in general.

4 Evaluation

I evaluate the research in terms of its validity, and I evaluate the programming project by contrasting the requirements in Section 2.5 with the actual work completed.

4.1 Results and Findings

In the first experiment, I perform ≈ 1200 simulation runs for each recruiter and condition. All recruiters (listed in Section 3.3) achieved non-trivial accuracy across all conditions (listed in Section 3.2):

Condition → Recruiter ↓	1) Include Protected	2) Omit Protected	3) Include 5 Proxy	4) Omit Distant Proxy
Random Forest	0.83	0.83	0.82	0.82
Bayesian	0.81	0.82	0.81	0.81
Shallow MLP	0.80	0.81	0.80	0.80
Logistic Regression	0.78	0.79	0.79	0.78
SVM	0.78	0.78	0.78	0.78
Encoder-Only Transformer	0.76	0.76	0.76	0.75
Deep MLP	0.74	0.75	0.74	0.74

Figure 4.1: Accuracy by recruiter by condition

In the second experiment, I run the simulation with only the most accurate recruiter (the random forest) across all 11 post-training mitigations (listed in Section 3.5) for ≈ 4000 simulation runs.

Space constraints limit a full discussion of the data generated by this simulation, so this is a small sample of results. A fuller and less technical summary of these findings is available here modelling-bias.com/walkthrough/the_results.

The simulated nature of this experiment means that any quantitative results are going to have limited applicability. Instead, I focus on qualitative findings, each of which relies on a careful set of assumptions which I provide in Section 4.2.

Throughout, I prefer the metrics derived from equalised odds (FNR difference and FPR difference), instead of those derived from predictive parity (justified in Section 2.2.2). I have omitted reference to significance testing for space reasons, but all qualitative results are statistically significant. Because this experiment involved a lot of simulated data, significance levels are very low ($<1e-10$).

1. Bias Often Emerges, even with an Unbiased Training Set

If data were biased it would mean that labels or features are inaccurate, or that applicants aren't randomly sampled from the distribution. In my simulation, the input data is a true reflection of the simulated distribution.

Across all (unmitigated) recruiters and conditions, the measurement of bias was likely to be non-negligible. Only 21% of recruiters demonstrated decisions with an absolute FNR difference of less than 5 percentage points.

A recruiter with 100% accuracy would exhibit no bias, except in demographic parity, so a reasonable objection to this finding is that it is due to the suboptimal accuracy of the average recruiter. However, if I constrain my analysis only to accurate recruiters (accuracy > 95%), I still find that only 46% of recruiters demonstrated decisions with an absolute FNR difference of less than 5 percentage points.

A final objection is that the result is due to competence rate differences between groups, which may not exist in a real-world scenario. Of the 245 runs with absolute competence rate difference < 0.05 and with accuracy > 95%, I still find that only 57% of recruiters make decisions with an absolute FNR difference of less than 5 percentage points:

Recruiter Group → Unbiasedness Criterion ↓	All Recruiters	Recruiters with accuracy >0.95	Recruiters with accuracy >0.95 and absolute competence rate difference <0.05
Absolute FNR Difference <0.05	21%	46%	57%
Absolute FPR Difference <0.05	21%	45%	60%
Absolute FDR Difference <0.05	27%	48%	58%
Absolute FOR Difference <0.05	27%	53%	61%
Absolute Hiring Rate Difference <0.05	15%	12%	77%

Figure 4.2: Proportion of recruiters in a given group satisfying some given criterion for unbiasedness

These are quite lenient criteria for unbiasedness, and in most conditions only the minority of recruiters satisfy them. So I conclude that bias is likely to emerge, regardless of whether it exists in the training set.

2. Standard Fairness Metrics often Disagree

From a set of scatter graphs, it is clear that the relationship between either of FNR difference and FPR difference and either of FDR difference or FOR difference is weak. If these measurements showed high agreement, we would expect to see all the points clustered around the line of best fit. Instead, they form a cloud, and it would be difficult to accurately predict one bias metric from the other:

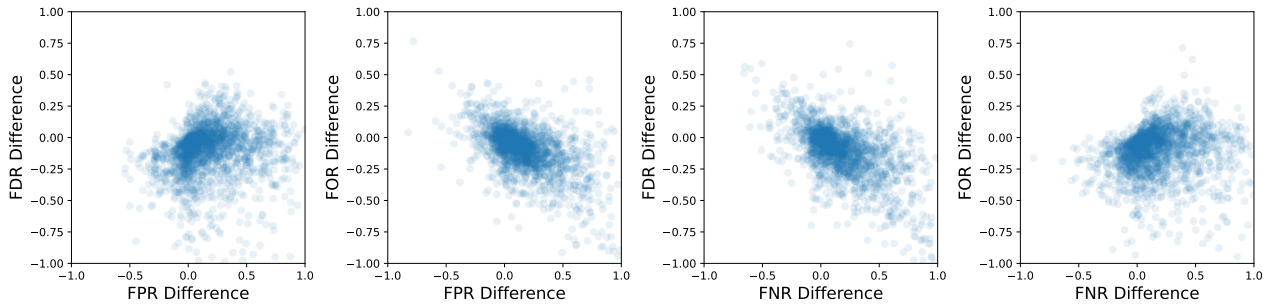


Figure 4.3: Scatter graph comparing each bias metric for each recruiter

When the competence rate difference between groups is high, I find that these metrics usually contradict each other on which bias the group is against. I plot the same scatter graphs for the case when absolute competence rate difference is greater than 30%. For each graph, each recruiter in the yellow quadrant produces decisions such that the measurements on the x-axis and y-axis disagree as to which group the recruiter favours:

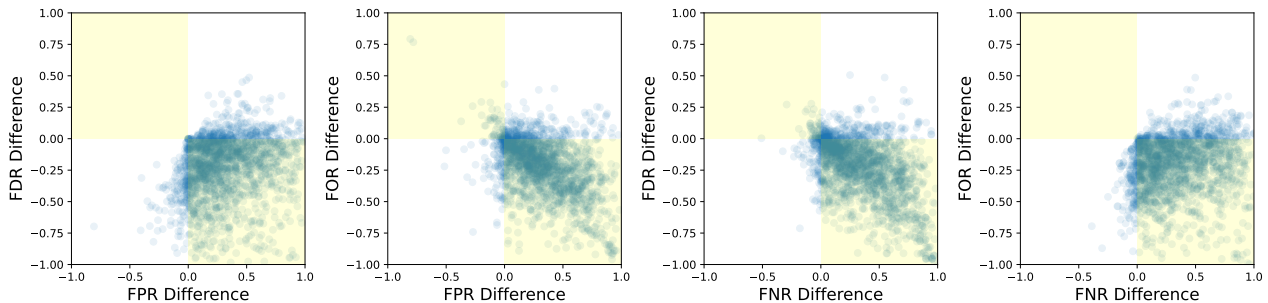


Figure 4.4: The same scatter graph for recruiters with a high absolute competence rate difference

Therefore, the criteria for bias show poor agreement in general. I elaborate on when they might disagree in the next result (Result 4.1.3).

3. Bias Tends to Exacerbate Existing Group Differences

If the competence difference is high, then FNR difference, FPR difference, and hiring rate difference (derived from the demographic parity and equalised odds criteria) are much more likely to measure the bias as being in favour of the more competent group. However, FDR difference and FOR difference (derived from predictive parity) are more likely to measure it in favour of the less competent group.

I demonstrate this with a histogram for each bias metric. A positive value for bias is in favour of the group with the higher competence rate, and a negative value for a bias measurement is in favour of the group with the lower competence rate. As before, I limit the recruiters measured here to where the competence difference is greater than 30%:

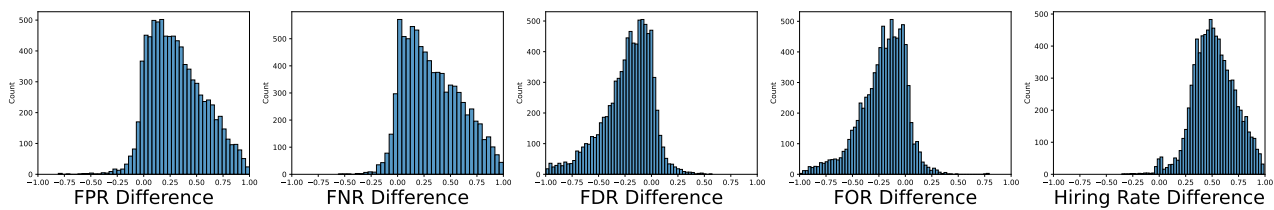


Figure 4.5: The number of recruiters exhibiting each level of bias for high competence differences

So it seems that when one group has lower average competence, it is likely to face unfair treatment. However, by only considering applicants who were hired, or those who weren't, we are likely to conclude that the disadvantaged group is favoured.

4. Masking Protected Characteristics Reduces Bias

The conditions I introduced in Section 3.2 do lead to significant differences in bias. For metrics inspired by equalised odds, omitting all distant proxy characteristics leads to the least bias, and revealing the protected characteristic directly or through proxy characteristics leads to the most bias.

However, for metrics inspired by predictive parity, less bias is exhibited when the protected characteristic is included. Omitting all distant proxy characteristics leads to the most bias.

I demonstrate these two phenomena with a set of KDE plots for bias across conditions for each metric. Where the line is higher for a particular value of bias, there are more recruiters which measure this value. Also note that a higher line at the centre of the graph implies that recruiters are less biased - the bias is more likely to be close to zero.

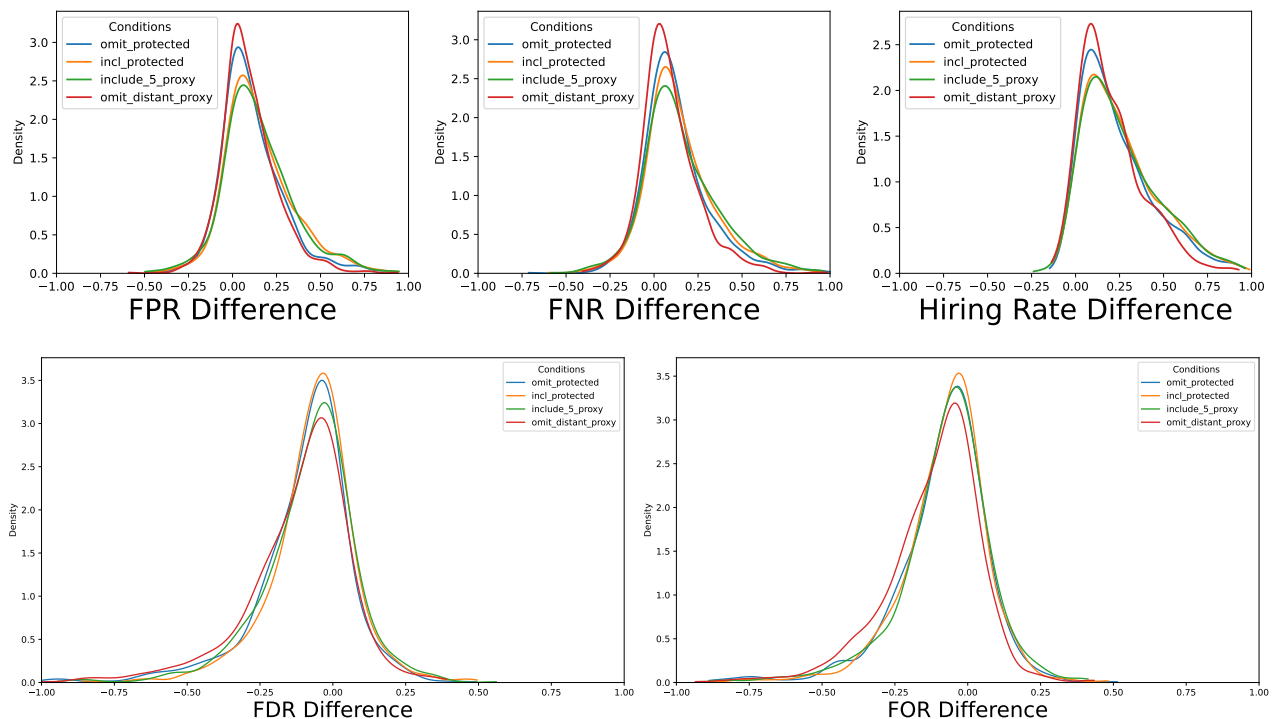


Figure 4.6: KDE plots for exhibited bias by condition, for each bias metric.

Condition	Avg. Abs. FNR Difference	Avg. Abs. FPR Difference	Avg. Abs. FDR Difference	Avg. Abs. FOR Difference	Avg. Abs. Hiring Rate Difference
4) Omit Distant Proxies	0.13	0.13	0.15	0.14	0.21
2) Omit Protected	0.16	0.15	0.13	0.13	0.24
3) Include Proxies	0.18	0.18	0.13	0.12	0.26
1) Include Protected	0.19	0.18	0.12	0.12	0.26

Figure 4.7: Average bias exhibited by condition.

5. Model Choice Affects Bias

The random forest recruiter is consistently less biased than any other recruiter, and in general, the models show significant difference in the bias they exhibit:

Bias Metric → Recruiter ↓	Absolute FNR Difference	Absolute FPR Difference	Absolute FDR Difference	Absolute FOR Difference	Absolute Hiring Rate Difference
Random Forest	0.166	0.162	0.131	0.128	0.245
Bayesian	0.199	0.192	0.141	0.137	0.262
Deep MLP	0.210	0.205	0.173	0.166	0.252
Shallow MLP	0.231	0.224	0.152	0.145	0.281
SVM	0.237	0.228	0.165	0.159	0.272
Logistic Regression	0.238	0.232	0.157	0.152	0.283
Encoder-Only Transformer	0.272	0.263	0.181	0.175	0.298

Figure 4.8: Average Bias by Recruiter

We might ask whether these differences are only due to differences in accuracy between the models, but one of the least accurate models, the deep MLP, is relatively unbiased. I also separately confirm that there is little relationship between accuracy and bias when there is low competence disparity, but these differences in recruiters remain.

6. Post-Training Mitigations Reduce Bias to an Extent

With statistical significance, all post-training bias mitigations reduce the bias they are attempting to reduce. However, only the demographic parity mitigation is able to satisfy its optimisation criterion:

Bias Metric → Mitigation ↓	Absolute FNR Difference	Absolute FPR Difference	Absolute FDR Difference	Absolute FOR Difference	Absolute Hiring Rate Difference
No Mitigation	0.1555	0.1573	0.1303	0.1319	0.2415
Ensure Demographic Parity	0.1546	0.1577	0.2517	0.2513	0.0012
Satisfy Proportional Parity	0.1346	0.1350	0.1364	0.1396	0.2219
Optimise for FNR Equality	0.0340	0.1695	0.2008	0.1738	0.1567
Optimise for FPR Equality	0.1695	0.0357	0.1727	0.2017	0.1585
Optimise for FNR and FPR Equality	0.0840	0.0830	0.1863	0.1868	0.1441
Optimise for FDR Equality	0.3490	0.2860	0.0787	0.1680	0.3786
Optimise for FOR Equality	0.2856	0.3469	0.1649	0.0810	0.3747
Optimise for FDR and FOR Equality	0.2930	0.2953	0.1000	0.1049	0.3556

Figure 4.9: Average Bias by Recruiter Mitigation (for Random Forest Recruiter)

4.2 Assumptions Analysis

To determine whether the findings from the simulation generalise, I critically consider the assumptions under which the simulation is an accurate model of a real recruiting scenario. I summarise them here:

1. It is possible to model an applicant as a system of random variables.
2. It is possible to model an applicant distribution as a bayesian network.
3. Characteristics are all categorical.
4. An applicant's objective competence can be defined.
5. Each applicant has only a single protected characteristic.
6. Applicants don't lie or omit information.
7. Recruiters directly receive an applicant's competence, and receive a uniform random sample of applicants.
8. Each recruiter makes a binary decision on each applicant.
9. Equalised odds, demographic parity, and the metrics I derived from them are valid measures of fairness.

I elaborate on these and how they might limit the research, leaving a full assumptions analysis to the appendix (Appendix A.7):

My network generation process assumes that there is mutual information between a child and its parents, and that the network is sparsely connected, with each node having between 2 and 3 parents. This may not hold, although that's unlikely to invalidate the central findings.

I have also assumed that there is no bias in a training set. This assumption won't hold in practice. The training set is a disproportionate sample of applicants and will contain a systematic pattern of inaccuracies. However, this assumption is useful because it allows us to isolate the effect of algorithmic bias. It does mean the research is more limited; we can only obtain insights on this particular bias.

I constrained our focus to binary classification which simplifies bias measurement. In most instances, recruitment is binary, but this may be overly reductive, a concern I discuss in Section 5.2.

Finally, I only consider output bias, that is, bias which can be measured entirely by the decisions made. This approach may obscure some of the nuances within machine fairness. There are many alternate methods of defining bias, including by the intentionality of the classifier, the context in which the system is used, or the consequences of the system itself.

4.3 Research Implications

If the assumptions do hold, or if the simulation isn't so sensitive to each assumption that violating it affects the findings, then I can generalise my findings to the application of machine learning in real-world recruiting and similar human classification tasks (such as loan approval or predicting recidivism). For each finding, I discuss implications:

1. Bias often emerges, even with an unbiased training set.

This conflicts with a standard explanation for why bias exists: that it is propagated from biased training sets and biased algorithmic design.^{[14][15]} While this may be the case, it is not a complete explanation of bias.

This means that bias must be considered throughout the development process of an ML model, advice which isn't always adhered to in typical working practice.^[106] In some ML projects, bias is only considered to the extent that the system meets a legal requirement.^[107]

There are further philosophical insights to this result. If we consider that an ML decision is simply the result of some optimisation process to make accurate decisions and that human decision-making resulted from a similar optimisation process, then this may be a partial explanation as to why bias exists in society.

2. Standard fairness metrics often disagree.

This is a finding I initially didn't expect, although it is supported by some existing research.^[108] In some cases, the choice of which bias metric to use is made without full understanding of the differences between fairness criteria,^{[19][109]} and some state-of-the-art models have been developed with what I would consider to be the wrong choice.^[110]

In fact, predictive parity is the dominant method for assessing bias in many domains, including in recidivism,^[39] where I don't think it is the appropriate criterion to use.^[41] Given that equalised odds and predictive parity are ostensibly similar, I suspect that the choice to use predictive parity over equalised odds was not carefully scrutinised. So this finding has useful implications for the field: models presumed to be fair may in fact exhibit bias.

3. Bias tends to exacerbate existing group differences.

When one group has lower average competence, it is likely to face unfair treatment. By considering hiring rate difference, FNR difference or FPR difference, one is likely to conclude that the lower-competence group is disadvantaged.

However, by only considering applicants who were hired, or only those who were rejected, one is likely to conclude that the disadvantaged group is instead favoured. FDR Difference and FOR Difference often exhibit values suggesting the recruiter favours the disadvantaged group.

Being able to predict the direction of bias allows us to estimate the effects of systems we know to be biased. We may have limited influence to debias existing algorithms, but understanding the consequences of those systems is useful.

This finding also supports an existing concern.^[111] Bias can be self-perpetuating. If bias is exacerbated by ML recruiters, causing the advantaged group to have a higher chance of receiving a job, and subsequently improving the skills of that advantaged group, then over time, we can expect there to be a pervasive and increasing bias in favour of that group. In fact, due to the discrepancy in

bias metrics (where considering outcomes appears to favour the disadvantaged group), recruiters may develop negative perceptions of the disfavoured group, amplifying bias further.

4. **Masking protected characteristics reduces bias.**

Masking just the protected characteristic reduces bias, and masking all the proxy characteristics (those which are closely related to the protected characteristic) reduces it further.

This is a simple way in which to mitigate bias, which somewhat validates the standard practice of hiding applicant names and identifying information within interviews and resumes.^[112] Determining which characteristics are proxy characteristics isn't computationally infeasible, and while there is research on the potential of removing proxy characteristics from model input,^[113] I was unable to find examples of this method in practice. Additionally, some dispute that this practice does reduce bias, claiming that increasingly unintuitive proxies are found instead.^[113] This research presents an alternative viewpoint to this perspective. Removing proxy characteristics does work, but only to an extent.

Finally, there are examples in practice of proxy characteristic masking with predictive parity being used as the bias metric.^{[114][115]} However, I have demonstrated that this masking worsens adherence to predictive parity.

5. **Model choice affects bias.**

Some recruiters are consistently less biased than others. For example, the random forest model exhibits lower average bias by all metrics, despite having only marginally higher accuracy. This aligns with the first finding, that bias exists with unbiased data. If it is some feature of the model which results in bias, we would expect the exhibited bias to vary with the model.

Accuracy has long been seen as the gold standard for a machine learning model, but its ability to exhibit fairness may be orthogonal to this measure. Researchers should pay attention to fairness, even during model choice.

6. **Post-training mitigations reduce bias to an extent.**

Adjusting group-specific thresholds after training can reduce bias without retraining a model. This is a simple way to reduce bias, and can be efficiently applied to existing systems, or those which weren't designed with bias in mind.

The mitigations to optimise predictive parity or equalised odds won't work as well in practice, because we don't have access to the ground truth of a mitigation set. Further, only demographic parity could be fully satisfied by such a mitigation. So post-training debiasing techniques are necessarily limited and for many problems, it isn't sufficient to consider bias only at the end of the machine learning process.

4.4 Evaluation against Requirements

I listed requirements in Section 2.5.

(1), (2), (3), (5), and (6) were achieved, as evidenced in the visualisation^[100] and Sections 3.1, 3.2, 3.4, and 3.5. It could be argued that the generated bayesian network is not truly random, because it is constrained to have low density high mutual information between each variable and its parents. However, I defended these criteria in 3.1.

(4) was achieved in Section 3.3, although it is debatable whether the ML techniques implemented are a fair cross-section of the state-of-the-art for classifying applicants. For example, generative LLMs have been used for this task by producing a generative summary resumes.^[30] It is difficult to fully inventory the state-of-the-art, because many proprietary algorithms are obscured.

(7) was achieved, as evidenced in Section 4.1 and the website analysis.^[116] The analysis of results was necessarily explorative so I was unable to robustly investigate every aspect of the data I collected. I justified in Section 4.3 why I think the insights I gathered were useful, with more research being required.

(8) was achieved and is available at modelling-bias.com/visualisation. (13) was also achieved.^{[101][117]}

I fulfil (9) to an extent by demonstrating that different recruiters exhibit consistently different levels of bias (Result 4.1.5). It is regrettable I wasn't able to investigate this more thoroughly. With more time, I would have investigated how varying each structural parameter of a recruiter changed the bias it exhibits. Time constraints prevented me from doing this.

I fulfil (10) to an extent by demonstrating that the four different application generation conditions lead to consistently different levels of bias (Result 4.1.4).

(11) and (12) are partially completed. I demonstrate that predictive parity and equalised odds disagree under certain circumstances (Result 4.1.2 and 4.1.3) and that post-mitigation techniques have limited efficacy (Result 4.1.6). More research is needed.

So requirements (1)-(8) and (13) were completed. Requirements (9)-(12) were partially completed.

5 Conclusions

5.1 Summary of Implications and Recommendations

Bias is likely to emerge even in the case where the training data is unbiased. Therefore, fairness must be considered throughout the entire machine learning process for a model to be fair.

Standard fairness metrics disagree and the wrong fairness metrics have been used in existing research and state-of-the-art practice. Models presumed to be fair may exhibit bias. The choice of whether to use predictive parity or equalised odds is a crucial one, and is contextual on the domain itself.

Bias exacerbates group differences, so we should expect the group with the lower competence rate to be further disadvantaged by a biased classifier. Measuring the bias within a system only by considering applicants who were hired and separately those who were rejected may imply the false conclusion that the model is biased in the opposite direction.

Masking protected variables and proxy variables is effective in improving adherence to equalised odds, but only to an extent. This mitigation reduces adherence to predictive parity. Model choice is relevant to bias and fairness may be an orthogonal consideration to accuracy in model selection.

While post-training bias mitigations reduce bias, they are only effective to an extent. Demographic parity can be fully satisfied in post-training, but this has some cost to accuracy.

5.2 Further Work

Many of the assumptions listed in Section 4.2 should be verified to hold in algorithmic recruiting. Randomly generated networks can be compared to real datasets in algorithmic recruiting. Many of the findings I present can also be verified in the real world, such as the disagreement between metrics inspired by predictive parity and metrics inspired by equalised odds. The effectiveness of post-training mitigations can also be empirically validated.

This research could be the starting point of an investigation into why bias is likely to exist in a machine-learning model, and more broadly any decision-making process, which does not rely on the assumption that bias exists in the training data.

I think one hypothesis demands further research. The requirement for a recruiter to categorically classify applicants may exacerbate or fully explain bias. Applicants who are near the threshold between categories are necessarily given a more extreme classification than they would have received in the continuous case.

Further support for this idea comes from the recent discourse around Hedden's thought experiment criticising existing bias metrics.^[118] He demonstrates how an "obviously fair" classifier, which guesses the next result of a person flipping a biased coin, is unfair by all bias metrics. It could be argued that this classifier isn't fair - specifically because it assigns a simplifying binary prediction to each person.^[119] It may be then that the categorical simplification critically limits this research and other research into fairness, so experimenting with the continuous case would be a useful extension.

5.3 Personal Lessons Learnt

I knew that the choice of this project carried some risk: there was no guarantee that I would find interesting results, I was mostly unaware of the different challenges that would present themselves, and the scope was very large, bringing together many areas of computer science.

That said, I'm really glad I chose it. My technical skills have improved, and my interest in machine fairness has made the project enjoyable throughout. I'm also really happy with the visualisation^[100] and walkthrough.^[101] That wasn't necessary for the submission of the dissertation, and it was a significant time commitment, but it's very rewarding to see non-computer scientists able to engage in this research. The intelligibility of the website meant I received much more feedback than I would have otherwise.

I've historically taken a different approach with coursework, often choosing to go for lower risk and more concrete projects. This research has been a useful reminder that that isn't always necessary. Indeed, this research wouldn't have been possible without leaving my comfort zone.

In retrospect, there are some things I wish I'd done differently:

- I should have chosen Python as a language sooner. I spent some time developing a prototype in Kotlin because I prefer its design as a language, but much of the system I finally implemented would have been impossible in this language. In the future, I will regard the existing ecosystem around a language as more important than my own design preference. Also, given that I knew I was going to host this on a virtual machine, I should have used Linux.
- I should have gathered data more quickly. I approached this project linearly: designing, then developing the visualisation, and only then running the simulation to gather data. If I had gathered data first, I would have been able to use that information to direct the rest of the project. This would have mitigated the risk I was unable to obtain useful data.
- I should have taken steps to constrain the scope of the project sooner. While I'm glad I didn't compromise on functionality, there were other scope-restricting decisions (such as only considering categorical networks and not continuous ones) I could have made sooner. Additionally, I shouldn't have developed the website until the simulation was complete, to further mitigate risk.

Overall, this has been a deeply rewarding experience. I've learnt a lot technically and gained confidence in my abilities. I think the results I present are surprising and actionable, and I hope that they will motivate further study.

Bibliography

- [1] 7 in 10 companies will use AI in the Hiring Process, Despite Most Saying it's Biased. *Resume-Builder.com*. 2024. URL: <https://www.resumebuilder.com/7-in-10-companies-will-use-ai-in-the-hiring-process-in-2025-despite-most-saying-its-biased/>.
- [2] *How AI is Deciding Who Gets Hired*. Bloomberg Originals. *www.youtube.com*. 2022. URL: <https://www.youtube.com/watch?v=6nGM37ThEsU>.
- [3] Cynthia Rudin, Caroline Wang, and Beau Coker. "The Age of Secrecy and Unfairness in Recidivism Prediction". In: *Harvard Data Science Review* 2.1 (Jan. 2020). DOI: <https://doi.org/10.1162/99608f92.6ed64b30>.
- [4] UK creating "murder prediction" tool to identify people most likely to kill. Vikram Dodd. *the Guardian*. 2025. URL: <https://www.theguardian.com/uk-news/2025/apr/08/uk-creating-prediction-tool-to-identify-people-most-likely-to-kill>.
- [5] Ana Cristina, Marcio Gomes, and Roberto Rigobon. "Algorithmic discrimination in the credit domain: what do we know about it?" In: *AI Society* 39 (May 2023), pp. 2059–2098. DOI: <https://doi.org/10.1007/s00146-023-01676-3>.
- [6] NHS to Launch World's Biggest Trial of AI Breast Cancer Diagnosis. Denis Campbell. *the Guardian*. 2025. URL: <https://www.theguardian.com/society/2025/feb/04/nhs-to-launch-worlds-biggest-trial-of-ai-breast-cancer-diagnosis>.
- [7] Nitin Kamble. "Product Recommendation System Using Machine Learning". In: *SSRN Electronic Journal* (2021). DOI: <https://doi.org/10.2139/ssrn.4245401>.
- [8] N Pavitha et al. "Movie Recommendation and Sentiment Analysis Using Machine Learning". In: *Global Transitions Proceedings* 3.1 (Apr. 2022). DOI: <https://doi.org/10.1016/j.gltp.2022.03.012>.
- [9] Eric Martin. "Search Engines: Applications of ML". In: *Encyclopedia of Machine Learning* (Dec. 2010), pp. 882–886. DOI: https://doi.org/10.1007/978-0-387-30164-8_744.
- [10] Ninareh Mehrabi et al. "A Survey on Bias and Fairness in Machine Learning". In: *ACM Computing Surveys* 54.6 (July 2021), pp. 1–35. DOI: <https://doi.org/10.1145/3457607>. URL: <https://dl.acm.org/doi/10.1145/3457607>.
- [11] Venkatesha Kurumayya. "Towards fair AI: a review of bias and fairness in machine intelligence". In: *Journal of Computational Social Science* 8.3 (May 2025). DOI: <https://doi.org/10.1007/s42001-025-00386-8>.
- [12] Solon Barocas, Moritz Hardt, and Arvind Narayanan. *Fairness in Machine Learning - Limitations and Opportunities*. 2018. URL: <https://fairmlbook.org/pdf/fairmlbook.pdf>.
- [13] Judea Pearl. *Causality: Models, Reasoning, and Inference*. 2nd. New York: Cambridge University Press, 2009. URL: <https://www.cambridge.org/core/books/causality/B0046844FAE10CBF274D4ACBDAEB5F5B>.
- [14] *Fairness: Types of Bias | Machine Learning Crash Course*. Anon. Google Developers. 2022. URL: <https://developers.google.com/machine-learning/crash-course/fairness/types-of-bias>.
- [15] *Research shows AI is often biased. here's how to make algorithms work for all of us*. Agbolade Omowole. *World Economic Forum*. 2021. URL: <https://www.weforum.org/stories/2021/07/ai-machine-learning-bias-discrimination/>.

- [16] *Researchers reduce bias in AI models while preserving or improving accuracy*. Adam Zewe. *MIT News | Massachusetts Institute of Technology*. 2024. URL: <https://news.mit.edu/2024/researchers-reduce-bias-ai-models-while-preserving-improving-accuracy-1211>.
- [17] *How AI can end bias* | SAP. SAP. *Sap.com*. 2025. URL: <https://www.sap.com/swiss/blogs/how-ai-can-end-bias>.
- [18] Michael A. Lones. “How to avoid machine learning pitfalls: a guide for academic researchers”. In: *arXiv:2108.02497 [cs]* (Aug. 2021). URL: <https://arxiv.org/abs/2108.02497>.
- [19] Caleb et al. “A Review of Fairness and A Practical Guide to Selecting Context-Appropriate Fairness Metrics in Machine Learning”. In: *arXiv (Cornell University)* (Nov. 2024). DOI: <https://doi.org/10.48550/arxiv.2411.06624>. URL: https://www.researchgate.net/publication/385721402_A_Review_of_Fairness_and_A_Practical_Guide_to_Selecting_Context-Appropriate_Fairness_Metrics_in_Machine_Learning.
- [20] Ricardo Trainotti Rabonato and Lilian Berton. “A systematic review of fairness in machine learning”. In: *AI and Ethics* (Sept. 2024). DOI: <https://doi.org/10.1007/s43681-024-00577-5>.
- [21] Susan Wharton Gates, Vanessa Gail Perry, and Peter M. Zorn. “Automated underwriting in mortgage lending: Good news for the underserved?” In: *Housing Policy Debate* 13.2 (Jan. 2002), pp. 369–391. DOI: <https://doi.org/10.1080/10511482.2002.9521447>.
- [22] Maryam Ahmed. “UK passport photo checker shows bias against dark-skinned women”. In: *BBC News* (Oct. 2020). URL: <https://www.bbc.co.uk/news/technology-54349538>.
- [23] Ziad Obermeyer et al. “Dissecting racial bias in an algorithm used to manage the health of populations”. In: *Science* 366.6464 (Oct. 2019), pp. 447–453. DOI: <https://doi.org/10.1126/science.aax2342>.
- [24] *Gender Shades*. Joy Buolamwini. *Gendershades.org*. 2018. URL: <http://gendershades.org/>.
- [25] Louis Lippens, Siel Vermeiren, and Stijn Baert. “The state of hiring discrimination A meta-analysis of (almost) all recent correspondence experiments”. In: *European Economic Review* 151 (Oct. 2022), p. 104315. DOI: <https://doi.org/10.1016/j.euroecorev.2022.104315>. URL: <https://www.sciencedirect.com/science/article/pii/S0014292122001957>.
- [26] Danielle Gaucher, Justin Friesen, and Aaron C. Key. “Evidence That Gendered Wording in Job Advertisements Exists and Sustains Gender Inequality”. In: *Journal of Personality and Social Psychology* 101.1 (2011), pp. 109–128. URL: <https://gap.hks.harvard.edu/evidence-gendered-wording-job-advertisements-exists-and-sustains-gender-inequality>.
- [27] Lincoln Quillian et al. “Meta-analysis of field experiments shows no change in racial discrimination in hiring over time”. In: *Proceedings of the National Academy of Sciences* 114.41 (Sept. 2017), pp. 10870–10875. DOI: <https://doi.org/10.1073/pnas.1706255114>. URL: <https://www.pnas.org/content/early/2017/09/11/1706255114/tab-article-info>.
- [28] *Employers: Preventing Discrimination*. GOV.UK. GOV.UK. 2024. URL: <https://www.gov.uk/employer-preventing-discrimination/recruitment>.
- [29] *Are Emily and Greg More Employable than Lakisha and Jamal? A Field Experiment on Labor Market Discrimination*. Marianne Bertrand and Sendhil Mullainathan. *National Bureau of Economic Research Working Paper Series*. 2003. URL: <https://www.nber.org/papers/w9873>.

- [30] *Are Emily and Greg Still More Employable than Lakisha and Jamal? Investigating Algorithmic Hiring Bias in the Era of ChatGPT*. Akshaj Kumar Veldanda et al. *arXiv.org*. 2023. URL: <https://arxiv.org/abs/2310.05135>.
- [31] Ashika Maharaj and Grace Oluwafunmilayo Obalade. “Artificial intelligence innovations in talent recruitment, retention, diversity mapping within South Africa: A meta narrative analysis”. In: *Journal of Infrastructure, Policy and Development* 9.2 (Mar. 2025), p. 10352. DOI: <https://doi.org/10.24294/jipd10352>. URL: https://www.researchgate.net/publication/389611934_Artificial_intelligence_innovations_in_talent_recruitment_retention_diversity_mapping_within_South_Africa_A_meta_narrative_analysis.
- [32] Layla Alhalwachi et al. “Impact of Artificial Intelligence and Machine Learning in Recruitment: Transformative Shift in the Traditional Methods of Hiring”. In: *Studies in big data* (Jan. 2024), pp. 117–124. DOI: https://doi.org/10.1007/978-3-031-73632-2_10.
- [33] *AI hiring tools may be filtering out the best job applicants*. Charlotte Lytton. *Bbc.co.uk*. 2024. URL: <https://www.bbc.co.uk/worklife/article/20240214-ai-recruiting-hiring-software-bias-discrimination>.
- [34] *Amazon Scraps Secret AI Recruiting Tool That Showed Bias against Women*. Jeffrey Dastin. *Reuters*. 2018. URL: <https://www.reuters.com/article/world/insight-amazon-scraps-secret-ai-recruiting-tool-that-showed-bias-against-women-idUSKCN1MKOAG/>.
- [35] Tim P. Morris, Ian R. White, and Michael J. Crowther. “Using simulation studies to evaluate statistical methods”. In: *Statistics in Medicine* 38.11 (Jan. 2019), pp. 2074–2102. DOI: <https://doi.org/10.1002/sim.8086>. URL: <https://onlinelibrary.wiley.com/doi/10.1002/sim.8086>.
- [36] Hailey R Banack, Eleanor Hayes-Larson, and Elizabeth Rose Mayeda. “Monte Carlo Simulation Approaches for Quantitative Bias Analysis: A Tutorial”. In: *Epidemiologic Reviews* (Oct. 2021). DOI: <https://doi.org/10.1093/epirev/mxab012>.
- [37] *Recidivism*: The percentage of former prisoners who are rearrested.
- [38] *How We Analyzed the COMPAS Recidivism Algorithm*. Jeff Larson et al. *ProPublica*. 2016. URL: <https://www.propublica.org/article/how-we-analyzed-the-compas-recidivism-algorithm>.
- [39] *Technical Response to Northpointe*. Julia Angwin Jeff Larson. *ProPublica*. 2016. URL: <https://www.propublica.org/article/technical-response-to-northpointe>.
- [40] Moritz Hardt, Eric Price, and Nathan Srebro. “Equality of Opportunity in Supervised Learning”. In: *arXiv:1610.02413 [cs]* (Oct. 2016). URL: <https://arxiv.org/abs/1610.02413>.
- [41] Alexandra Chouldechova. “Fair prediction with disparate impact: A study of bias in recidivism prediction instruments”. In: *arXiv:1610.07524 [cs, stat]* (Oct. 2016). URL: <https://arxiv.org/abs/1610.07524>.
- [42] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2020. ISBN: 9780134610993. URL: <https://aima.cs.berkeley.edu/>.
- [43] Paula A. Braveman et al. “Systemic and Structural Racism: Definitions, Examples, Health Damages, and Approaches to Dismantling”. In: *Health Affairs* 41.2 (Feb. 2022), pp. 171–178. DOI: <https://doi.org/10.1377/hlthaff.2021.01394>. URL: <https://www.healthaffairs.org/doi/10.1377/hlthaff.2021.01394>.
- [44] *Systematic Inequality*. Angela Hanks, Danyelle Solomon, and Christian Weller. *Center for American Progress*. 2018. URL: <https://www.americanprogress.org/article/systematic-inequality/>.

- [45] *The Importance of Immutability in Employment Discrimination Law*. Sharona Hoffman. *Ssrn.com*. 2010. URL: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=1680512.
- [46] *Python Software Foundation License*. Python Software Foundation. 2023. URL: <https://docs.python.org/3/license.html>.
- [47] *pgmpy License*. pgmpy Development Team. 2024. URL: <https://github.com/pgmpy/pgmpy/blob/dev/LICENSE>.
- [48] *pgmpy: Probabilistic Graphical Models using Python*. Abinash Ankan and Aditya Panda. 2015. URL: <https://github.com/pgmpy/pgmpy>.
- [49] Oriol Abril-Pla et al. “PyMC: a modern, and comprehensive probabilistic programming framework in Python”. In: *PeerJ Computer Science* 9 (2023), e1516. DOI: 10.7717/peerj-cs.1516. URL: <https://doi.org/10.7717/peerj-cs.1516>.
- [50] *NetworkX License*. NetworkX Developers. 2024. URL: <https://github.com/networkx/networkx/blob/main/LICENSE.txt>.
- [51] *scikit-learn License*. scikit-learn Developers. 2023. URL: <https://github.com/scikit-learn/scikit-learn/blob/main/LICENSE>.
- [52] *PyTorch License*. PyTorch Contributors. 2024. URL: <https://github.com/pytorch/pytorch/blob/master/LICENSE>.
- [53] *pandas License*. pandas Developers. 2024. URL: <https://github.com/pandas-dev/pandas/blob/main/LICENSE>.
- [54] *Matplotlib License*. Matplotlib Team. 2025. URL: <https://matplotlib.org/stable/users/project/license.html>.
- [55] *Seaborn License*. Michael Waskom. 2024. URL: <https://github.com/mwaskom/seaborn/blob/master/LICENSE>.
- [56] *SciPy License*. SciPy Developers. 2024. URL: <https://github.com/scipy/scipy/blob/main/LICENSE.txt>.
- [57] *TypeScript License*. Microsoft Corporation. 2025. URL: <https://github.com/microsoft/TypeScript/blob/main/LICENSE.txt>.
- [58] *W3C Document License*. World Wide Web Consortium. 2023. URL: <https://www.w3.org/copyright/document-license-2023/>.
- [59] *Svelte License*. Svelte Contributors. 2025. URL: <https://github.com/sveltejs/svelte/blob/master/LICENSE.md>.
- [60] *Tailwind CSS License*. Tailwind Labs. 2025. URL: <https://github.com/tailwindlabs/tailwindcss/blob/master/LICENSE>.
- [61] *Skeleton.dev License*. Skeleton Labs. 2025. URL: <https://github.com/skeletonlabs/skeleton/blob/main/LICENSE>.
- [62] *D3.js License*. Mike Bostock. 2024. URL: <https://github.com/d3/d3/blob/main/LICENSE>.
- [63] *Django License*. Django Software Foundation. 2025. URL: <https://www.djangoproject.com/foundation/license/>.
- [64] *GreenSock Animation Platform License*. GreenSock. 2025. URL: <https://greensock.com/license/>.
- [65] *GitHub Terms of Service*. GitHub, Inc. 2025. URL: <https://docs.github.com/en/site-policy/github-terms/github-terms-of-service>.

- [66] *Git License*. The Git Project. 2025. URL: <https://github.com/git/git/blob/master/COPYING>.
- [67] *Google Cloud Terms of Service*. Google LLC. 2025. URL: <https://cloud.google.com/terms>.
- [68] *Supabase License*. Supabase Community. 2025. URL: <https://github.com/supabase/supabase/blob/master/LICENSE>.
- [69] *PyCharm License Agreement*. JetBrains s.r.o. 2025. URL: <https://www.jetbrains.com/pycharm/terms/license.html>.
- [70] *IONOS General Terms and Conditions*. IONOS SE. 2025. URL: <https://www.ionos.co.uk/terms-gtc>.
- [71] *nginx License*. nginx, Inc. 2025. URL: <https://nginx.org/LICENSE>.
- [72] *Modelling Bias*. The Candidate. 2025. URL: <https://modelling-bias.com/>.
- [73] *Advanced Data Science - Cambridge*. 2024. URL: <https://www.cl.cam.ac.uk/teaching/2425/ADS/>.
- [74] *Deep Neural Networks - Cambridge*. 2024. URL: <https://www.cl.cam.ac.uk/teaching/2425/DNN/>.
- [75] *Github*. URL: <https://github.com/>.
- [76] David J. Anderson. *Kanban: Successful Evolutionary Change for Your Technology Business*. United States: Blue Hole Press, 2010. ISBN: 978-0984521401.
- [77] *Trump's anti-DEI battle threatens nonprofits trying to fill critical labor gaps*. Associated Press. 2025. URL: <https://apnews.com/article/dei-women-trades-construction-trump-chicago-058eb023e6d176f023886332fb0a5745>.
- [78] *UK Employment Law Coffee Break: Supreme Court Ruling on Equality Act Definitions and Employment*. Osborne Clarke. 2025. URL: <https://www.osborneclarke.com/insights/uk-employment-law-coffee-break-supreme-court-ruling-equality-act-definitions-employment>.
- [79] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. 2nd. Wiley-Interscience, 2006. ISBN: 978-0471241959.
- [80] *Graph Generators — networkx 3.3 documentation*. *networkx.org*. URL: <https://networkx.org/documentation/stable/reference/generators.html>.
- [81] Jack Kuipers and Giusi Moffa. “Uniform random generation of large acyclic digraphs”. In: *Statistics and Computing* 25.2 (Nov. 2013), pp. 227–242. DOI: <https://doi.org/10.1007/s11222-013-9428-y>.
- [82] R W Robinson. “Counting unlabeled acyclic digraphs”. In: *Lecture notes in mathematics* (Jan. 1977), pp. 28–43. DOI: <https://doi.org/10.1007/bfb0069178>.
- [83] Samuel Kotz, N. Balakrishnan, and Norman L. Johnson. *Continuous Multivariate Distributions, Volume 1: Models and Applications*. 2nd. Hoboken, New Jersey, USA: Wiley-Interscience, 2004. Chap. Chapter 49: The Dirichlet Distribution, pp. 599–614. ISBN: 0471697579.
- [84] Jiayu Lin. *On The Dirichlet Distribution*. Tech. rep. Department of Mathematics and Statistics, Queen's University, 2016. URL: <https://mast.queensu.ca/~communications/Papers/msc-jiayu-lin.pdf>.
- [85] *pgmpy*. 2025. URL: https://pgmpy.org/approx_infer/bn_sampling.html.
- [86] *Putting It All Together!* The Candidate. *Modelling-bias.com*. 2025. URL: https://modelling-bias.com/walkthrough/putting_it_all_together.

- [87] *RandomForestClassifier*. Scikit-Learn. *Scikit-learn.org*. 2018. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>.
- [88] *Support Vector Classifier*. scikit-learn. *Scikit-learn.org*. 2019. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>.
- [89] *Logistic Regression*. scikit-learn. *Scikit-learn.org*. 2014. URL: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.
- [90] *Maximum Likelihood Estimator - pgmpy*. *pgmpy*. 2025. URL: https://pgmpy.org/param_estimator/mle.html.
- [91] *Hill Climb Search - pmgpy*. *pgmpy.org*. URL: https://pgmpy.org/structure_estimator/hill.html.
- [92] *Linear Layer - PyTorch*. *Pytorch.org*. 2024. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Linear.html#torch.nn.Linear>.
- [93] *ReLU layer - PyTorch*. *Pytorch.org*. 2024. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.ReLU.html#torch.nn.ReLU>.
- [94] *Adam Optimiser - PyTorch*. *Pytorch.org*. 2024. URL: <https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html>.
- [95] *TransformerEncoder - PyTorch*. *Pytorch.org*. 2024. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.TransformerEncoder.html>.
- [96] *Dual Annealing - SciPy*. *docs.scipy.org*. URL: https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.dual_annealing.html.
- [97] *What is Compute Engine? - Google Cloud*. *Google Cloud Blog*. URL: <https://cloud.google.com/blog/topics/developers-practitioners/what-compute-engine-use-cases-security-pricing-and-more>.
- [98] *Supabase - The Open Source Firebase Alternative*. Supabase. *Supabase*. URL: <https://supabase.com/>.
- [99] *Project Jupyter*. Project Jupyter. *Jupyter.org*. 2019. URL: <https://jupyter.org>.
- [100] *Modelling Bias - Visualisation*. The Candidate. 2025. URL: <https://modelling-bias.com/visualisation>.
- [101] *Modelling Bias - Visualisation*. The Candidate. 2025. URL: <https://modelling-bias.com/walkthrough>.
- [102] *Architectural Styles and the Design of Network-based Software Architectures*. Roy Thomas Fielding. 2000. URL: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [103] *The WebSocket Protocol*. Ian Fette and Alexey Melnikov. 2011. URL: <https://www.rfc-editor.org/rfc/rfc6455.html>.
- [104] *Bayes Server - Online Demo*. *www.bayesserver.com*. URL: <https://online.bayesserver.com/>.
- [105] *D3.js - Data-Driven Documents*. Mike Bostock. *D3js.org*. 2020. URL: <https://d3js.org/>.
- [106] Julia Dressel and Hany Farid. "The accuracy, fairness, and limits of predicting recidivism". In: *Science Advances* 4.1 (Jan. 2018). DOI: <https://doi.org/10.1126/sciadv.aao5580>. URL: <https://advances.sciencemag.org/content/advances/4/1/eaao5580.full.pdf>.
- [107] *Fairness in Machine Learning: Regulation or Standards?* Brookings. URL: <https://www.brookings.edu/articles/fairness-in-machine-learning-regulation-or-standards/>.

- [108] *Fairness in Criminal Justice Risk Assessments: The State of the Art*. Richard Berk et al. *arXiv.org*. 2017. URL: <https://arxiv.org/abs/1703.09207>.
- [109] *Measuring Non-Expert Comprehension of Machine Learning Fairness Metrics*. Debjani Saha et al. *Icml.cc*. 2020. URL: <https://icml.cc/virtual/2020/poster/5977>.
- [110] Angelina Wang et al. “AI’s Fairness Problem: When Treating Everyone the Same is the Wrong Approach”. In: *Stanford Law School* (Feb. 2025). URL: <https://law.stanford.edu/press/ais-fairness-problem-when-treating-everyone-the-same-is-the-wrong-approach/>.
- [111] Fadlul Musrifah and Ika Ariani Hasanah. “Ethical Implications of AI-Driven Recruitment: A Multi-Perspective Study on Bias and Transparency in Digital Hiring Platforms”. In: *Journal of Management and Informatics* 4.1 (Apr. 2025), pp. 599–616. DOI: 10.51903/jmi.v4i1.140. URL: <https://jmi.stekom.ac.id/index.php/jmi/issue/view/10>.
- [112] Sean Fath, Richard P Larrick, and Jack B Soll. “Encouraging self-blinding in hiring”. In: *Behavioral science policy* 9.1 (Apr. 2023), pp. 45–57. DOI: <https://doi.org/10.1177/23794607231192721>.
- [113] Anya Prince et al. “Proxy Discrimination in the Age of Artificial Intelligence and Big Data”. In: 105 (), p. 1257. URL: <https://ilr.law.uiowa.edu/sites/ilr.law.uiowa.edu/files/2022-10/Proxy%20Discrimination%20in%20the%20Age%20of%20Artificial%20Intelligence%20and%20Big%20Data%20.pdf>.
- [114] *When blind hiring advances DEI — and when it doesn’t*. Sean Fath. *Harvard Business Review*. 2023. URL: <https://hbr.org/2023/06/when-blind-hiring-advances-dei-and-when-it-doesnt>.
- [115] Kewen Peng, Joymallya Chakraborty, and Tim Menzies. “FairMask: Better Fairness via Model-based Rebalancing of Protected Attributes”. In: *IEEE Transactions on Software Engineering* (2022), pp. 1–14. DOI: <https://doi.org/10.1109/TSE.2022.3220713>. URL: <https://ieeexplore.ieee.org/document/9951398>.
- [116] *Modelling Bias - The Results*. The Candidate. 2025. URL: https://modelling-bias.com/walkthrough/the_results.
- [117] *Modelling Bias - Guide*. The Candidate. 2025. URL: <https://modelling-bias.com/guide>.
- [118] Brian Hedden. “On statistical criteria of algorithmic fairness”. In: *Philosophy Public Affairs* 49.2 (Mar. 2021), pp. 209–231. DOI: <https://doi.org/10.1111/papa.12189>.
- [119] Anders Søgaard, Klemens Kappel, and Thor Grünbaum. “On Hedden’s proof that machine learning fairness metrics are flawed”. In: *Inquiry* (Feb. 2024), pp. 1–20. DOI: <https://doi.org/10.1080/0020174x.2024.2315169>.
- [120] Leslie Lamport. *LaTeX: A Document Preparation System*. 2nd. Reading, Massachusetts: Addison-Wesley, 1994. ISBN: 978-0201529838.
- [121] Michael Zedler. *The Fourier-GUTenberg Math Fonts*. Available at <http://www.ctan.org/pkg/fourier>. 2004.
- [122] *Roboto*. Christian Robertson. 2011. URL: <https://fonts.google.com/specimen/Roboto>.
- [123] *Bootstrap Icons*. The Bootstrap Authors. 2021. URL: <https://icons.getbootstrap.com/>.

A Appendices

A.1 Assumed Preliminaries

Binary Classification Terminology

In the context of binary classification in recruitment:

- A **True Positive** (TP) is a qualified candidate who was hired.
- A **False Positive** (FP) is an unqualified candidate who was hired.
- A **False Negative** (FN) is a qualified candidate who was rejected.
- A **True Negative** (TN) is an unqualified candidate who was rejected.

And within a particular group:

- **False Negative Rate** (FNR) is the proportion of qualified applicants who were rejected.
- **False Positive Rate** (FPR) is the proportion of unqualified applicants who were hired.
- **False Omission Rate** (FOR) is the proportion of rejected applicants who were qualified.
- **False Discovery Rate** (FDR) is the proportion of hired applicants who were unqualified.

Where, for example, FN refers to the count of false negatives within a group, these are derived as follows:

$$\text{FNR} = \frac{\text{FN}}{\text{FN} + \text{TP}} \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} \quad \text{FOR} = \frac{\text{FN}}{\text{FN} + \text{TN}} \quad \text{FDR} = \frac{\text{FP}}{\text{FP} + \text{TP}}$$

These measures can be calculated for all applicants or for a subset of applicants.

Directed Acyclic Graphs

A directed acyclic graph (DAG) is a tuple (V, E) where $V = \{v_1, \dots, v_n\}$ is a finite set of nodes, and $E \subseteq V \times V$ is a set of directed edges between nodes. If $(v_i, v_j) \in E$ then there is a directed edge from node v_i to node v_j , denoted $v_i \rightarrow v_j$.

A node's parents is the set of nodes with an edge to it and a node's neighbours is the set of nodes with any edge to it:

$$\text{Parents}(v_i) = \{v \in V \mid v \rightarrow v_i\} \quad \text{Neighbours}(v_i) = \{v \in V \mid v \rightarrow v_i \vee v_i \rightarrow v\}$$

A cycle is a non-empty path of the form $v_i \rightarrow \dots \rightarrow v_i$, where the path starts and ends on the same node. A DAG is a directed graph with no cycles.

A DAG is weakly connected if for every pair of nodes $v_i, v_j \in V$, there exists a sequence of edges connecting them $v_i \leftrightarrow \dots \leftrightarrow v_j$, where $\leftrightarrow$ denotes $\leftarrow$ or $\rightarrow$.

A.2 Calculating $c_{m,s,k}$

$$c_{m,s,k} = (t_m)^k - d_{m,s,k}$$

- $c_{m,s,k}$ is the number of DAGs which can be built by adding k outpoints to some fixed DAG with m nodes and s outpoints. For the remainder of this section we refer to each of those k outpoints as a new outpoint, and each of the s outpoints in the fixed subgraph as an old outpoint. $c_{m,s,k}$ is the number of valid ways in which new outpoints can be linked to nodes in the fixed subgraph.
- t_m is the number of ways in which a specific new outpoint can be linked to nodes in the fixed subgraph:

$$t_m = \sum_{i=\text{min-parents}}^{\text{max-parents}} \binom{m}{i}$$

- $d_{m,s,k}$ is the number of ways we could link all k new outpoints to all m nodes in the fixed subgraph such that one of the s old outpoints wasn't linked to a new outpoint. We must discount this from $c_{m,s,k}$ because the associated DAGs have more than k outpoints (these old outpoints are also new outpoints), and so are invalid.

In order to calculate $d_{m,s,k}$, we might start by calculating the number of ways to link old outpoints to new outpoints such that a specific old outpoint is disconnected from all new outpoints:

$$\left(\sum_{x=\text{min-parents}}^{\text{max-parents}} \binom{m-1}{x} \right)^k$$

- $m-1$ is the size of the set of old outpoints, excluding the one to be disconnected.
- There are k outpoints to link.

And then multiply this by the number of old outpoints which could be disconnected from new outpoints:

$$s \cdot \left(\sum_{x=\text{min-parents}}^{\text{max-parents}} \binom{m-1}{x} \right)^k$$

However, this approach overcounts configurations. For example, the configuration where two specific old outpoints are disconnected from all new out-points is counted twice, once for the first old outpoint and again for the second.

To fix this, we use the inclusion-exclusion principle. We discount those overcounted configurations by alternating between subtracting and adding terms for how many specific out-points could be disconnected. Then:

$$d = \sum_{i=1}^s (-1)^{i+1} \binom{s}{i} \left(\sum_{x=\text{min-parents}}^{\text{max-parents}} \binom{m-i}{x} \right)^k$$

And:

$$c_{m,s,k} = \left(\sum_{i=\text{min-parents}}^{\text{max-parents}} \binom{m}{i} \right)^k - \sum_{i=1}^s (-1)^{i+1} \binom{s}{i} \left(\sum_{x=\text{min-parents}}^{\text{max-parents}} \binom{m-i}{x} \right)^k$$

A.3 An ML Task Where Predictive Parity is a Suitable Metric

Consider a system where two groups of patients are screened for a very rare disease and false negative rate should be very low. If prevalence differs significantly between those groups, failing to satisfy predictive parity may mean that for one group the test is very reliable, and for the other it isn't. That may cause a large disparity in the amount of resources which go to either group. In this case, satisfying equalised odds means that a person who doesn't have the disease is just as likely to be flagged for either group, which isn't a particularly desirable criterion.

A.4 Recruiter Specifications

Random Forest

```
def train(self, application_train: pd.DataFrame, score_train: pd.Series):
    self.model.fit(application_train, score_train)

def predict_scores(self, applications: pd.DataFrame) -> pd.Series:
    return pd.Series(self.model.predict_proba(applications)[: , 1])
```

Support Vector Machine

```
def train(self, application_train: pd.DataFrame, score_train: pd.Series):
    self.model.fit(application_train, score_train)

def predict_scores(self, applications: pd.DataFrame) -> pd.Series:
    return pd.Series(self.model.decision_function(applications))
```

Logistic Regression

```
def train(self, application_train: pd.DataFrame, score_train: pd.Series):
    self.model.fit(application_train, score_train)

def predict_scores(self, applications: pd.DataFrame) -> pd.Series:
    return pd.Series(self.model.decision_function(applications))
```

Bayesian

```
def train(self, application_train: pd.DataFrame, score_train: pd.Series):
    data = application_train.copy()
    data["score"] = score_train
    hc = HillClimbSearch(data)
    best_model = hc.estimate(scoring_method=BicScore(data))
    self.model = BayesianNetwork(best_model.edges())
    self.model.fit(data, estimator=MaximumLikelihoodEstimator)

def predict_scores(self, applications: pd.DataFrame) -> pd.DataFrame:
    infer = VariableElimination(self.model)
    probabilities = [
        infer.query(variables=["score"], evidence=row.to_dict()).values
        for _, row in applications.iterrows()
    ]
    return pd.DataFrame(probabilities,
        columns=self.model.get_cpds("score").state_names["score"])[1]
```

Deep Multi-Layer Perceptron

```
class DeepMLP(nn.Module):
    def __init__(self, input_size: int, hidden_size: int = 16, depth: int = 16):
        super(DeepMLP, self).__init__()
        layers = [nn.Linear(input_size, hidden_size), nn.ReLU()]
        for _ in range(depth - 1):
            layers += [nn.Linear(hidden_size, hidden_size), nn.ReLU()]
        layers += [nn.Linear(hidden_size, 1)]
        self.model = nn.Sequential(*layers)

    def forward(self, x):
        return self.model(x)

class DeepMLPRecruiter(Recruiter):
    def train(self, application_train: pd.DataFrame, score_train: pd.Series):
        self.model = DeepMLP(len(application_train.columns))
        self.optimiser = optim.Adam(self.model.parameters(), lr=0.01)
        train_nn_model(self.model, self.optimiser, application_train,
                        score_train, batch_size = 512, epochs = 10)

    def predict_scores(self, applications: pd.DataFrame) -> pd.Series:
        self.model.eval()
        X = torch.tensor(applications.values, dtype=torch.float32)
        return pd.Series(self.model(X).detach().numpy().flatten())
```

Shallow Multi-Layer Perceptron

```
class ShallowMLP(nn.Module):
    def __init__(self, input_size: int, hidden_size: int = 64):
        super(ShallowMLP, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, 1)
        )

    def forward(self, x):
        return self.model(x)

class ShallowMLPRecruiter(Recruiter):
    def train(self, application_train: pd.DataFrame, score_train: pd.Series):
        self.model = ShallowMLP(len(application_train.columns))
        self.optimiser = optim.Adam(self.model.parameters(), lr=0.01)
        train_nn_model(self.model, self.optimiser, application_train,
                        score_train, batch_size = 512, epochs = 4)

    def predict_scores(self, applications: pd.DataFrame) -> pd.Series:
        self.model.eval()
        X = torch.tensor(applications.values, dtype=torch.float32)
        return pd.Series(self.model(X).detach().numpy().flatten())
```

Encoder-Only Transformer

```

class Transformer(nn.Module):
    def __init__(self, input_size: int, hidden_size: int = 16,
                  depth: int = 4, num_heads: int = 2, dropout: float = 0.1):
        super(Transformer, self).__init__()
        self.first_layer = nn.Linear(input_size, hidden_size)
        self.encoder = nn.TransformerEncoder(
            nn.TransformerEncoderLayer(d_model=hidden_size,
                                       nhead=num_heads, dropout=dropout, batch_first=True),
            num_layers=depth
        )
        self.output_layer = nn.Linear(hidden_size, 1)

    def forward(self, x):
        first_layer_output = self.first_layer(x)
        encoder_output = self.encoder(first_layer_output)
        return self.output_layer(encoder_output)

class EncoderOnlyTransformerRecruiter(Recruiter):
    def train(self, application_train: pd.DataFrame, score_train: pd.Series):
        self.model = Transformer(len(application_train.columns))
        self.optimiser = optim.Adam(self.model.parameters(), lr=1e-4)
        train_nn_model(self.model, self.optimiser, application_train,
                       score_train, batch_size = 512, epochs = 3)

    def predict_scores(self, applications: pd.DataFrame) -> pd.Series:
        self.model.eval()
        X = torch.tensor(applications.values, dtype=torch.float32)
        return pd.Series(self.model(X).detach().numpy().flatten())

```

A.5 Conceptualising Bias Metrics

To conceptualise these measurements I introduce the notion of **Relative Standing within Group**. The relative standing within group A of some applicant is the proportion of group A who are assigned a lower score.

Hiring Rate Difference

$$\text{Hiring Rate Difference} = \text{Hiring Rate}_A - \text{Hiring Rate}_B$$

It might seem that a positive Hiring Rate Difference is the likelihood of an applicant from group A being hired, where they would have been rejected if they were from group B. However, this intuition is only correct under the counterfactual interpretation that an applicant being from a different group would change all their other characteristics, dependent on the underlying applicant distribution. Conceptualising hiring rate difference in this way carries the false implication that it denotes how hiring likelihood is affected by performing a localised intervention to the protected characteristic. But, this is inaccurate. A recruiter may not even have access to that characteristic, meaning intervening to change it would have no effect on its decision.

Instead, we can conceptualise a positive Hiring Rate Difference as follows: If we consider some random applicant from group B, this is the probability that they are rejected and that an applicant from group A, with the same relative standing within their own protected group, is hired instead.

FNR Difference and FPR Difference

$$\text{FNR Difference} = \text{False Negative Rate}_B - \text{False Negative Rate}_A$$

$$\text{FPR Difference} = \text{False Positive Rate}_A - \text{False Positive Rate}_B$$

We can conceptualise a positive FNR Difference as follows: If we consider some random qualified applicant from group B, this is the probability that they are rejected and that a qualified applicant from group A, with the same relative standing within the qualified members of their own protected group is hired instead (and vice versa for a negative FNR Difference).

We can conceptualise a positive FPR Difference as follows: If we consider some random unqualified applicant from group A, this is the probability that they are accepted and that an unqualified applicant from group B, with the same relative standing within the unqualified members of their own protected group, is hired instead (and vice versa for a negative FPR Difference).

FDR Difference and FOR Difference

$$\text{FDR Difference} = \text{False Discovery Rate}_A - \text{False Discovery Rate}_B$$

$$\text{FOR Difference} = \text{False Omission Rate}_B - \text{False Omission Rate}_A$$

FDR Difference is the absolute difference in competence rates between protected groups within the pool of hired applicants, and FOR Difference is the same difference within the pool of rejected applicants.

A.6 Data Analysis Example

Pre-Processing

I decided that throughout my exploration, it would be useful to investigate whether each finding holds if there is a low competence rate difference between groups.

So I filter the dataset for this purpose, and build functions which accept any dataframe:

```
runs["c_diff"] = (runs["competent_rate_1"] - runs["competent_rate_2"]).abs()
rec_runs = runs[runs["condition"] != 5]
very_low_disparity = rec_runs[rec_runs["c_diff"] < 0.01]

def compare(col_name_1, col_name_2, df = rec_runs, line = True, max_points=200):
    df_sample = df.sample(n=min(len(df), max_points))
    plt.scatter(df_sample[col_name_1], df_sample[col_name_2], alpha=0.1)
```

Exploration

With this, I noticed that there was a higher correlation between bias metrics at low competence rate differences. Before exploring this too quickly, I wanted to create a quick visualisation of the correlations between metrics, the variance from the line of best fit, and the significance of these correlations for each bias pairing.

This is the exploratory stage, so I wasn't too concerned with code quality or visualisation quality. The aim is to quickly experiment:

```
for i, (metric1, metric2) in enumerate(itertools.combinations(
    ["fpr_disp_adj", "fnr_disp_adj", "fdr_disp_adj", "for_disp_adj"], 2)):
    plt.subplot(3, 4, 2 * i + 1)
    compare(metric1, metric2, df = low_disparity)
    plt.subplot(3, 4, 2 * i + 2)
    plot_residuals(metric1, metric2, df = low_disparity)
```

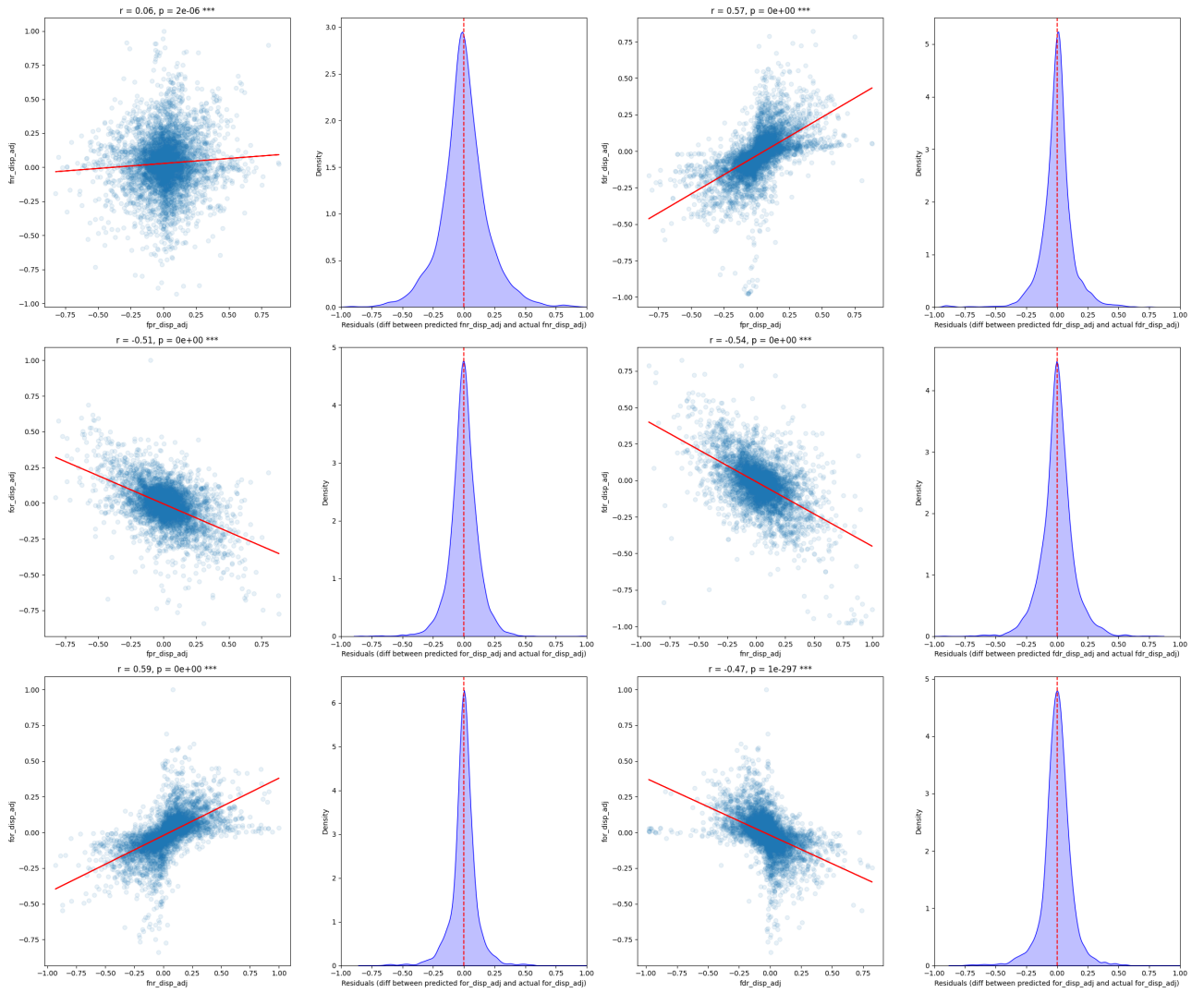


Figure A.1: Comparing each bias metric, displaying its variance, and the significance of the correlation for low competence rate disparity

Addressing Questions

After additional exploration, I was able to use this finding to isolate that certain bias metrics tend to disagree when competence difference is high. I then put thought into the best way to represent this information.

The result of that process is Result 2 in modelling-bias.com/walkthrough/the_results or Result 4.1.2 in this document.

A.7 Assumptions Analysis

1. It is possible to model an applicant as a system of random variables

Representing a complex system as a collection of variables is a common abstraction. However, we can only model a limited number of characteristics, and so I assume that our results generalise to scenarios involving many more.

One possible explanation for bias is that it arises from the inability of some ML methods to model complex dependencies between variables. Being unable to represent a large number of random variables may limit our ability to test this theory. However, I still verify it to an extent by testing simpler models. Indeed, I do find that simpler models (such as the linear model) generally perform worse. So, overall, this assumption does not seem limiting.

2. It is possible to model an applicant distribution as a bayesian network

This was justified in Section 2.3.1. If we assume that an applicant is a system of random variables, there will be some bayesian network that can represent it.

However, our network generation process also assumes that there is some mutual information between a child and its parents, and that the network is sparsely connected in general, with each node having between 2 and 3 parents. This is unlikely to hold in practice.

In fact, this may be a minor limitation of the research: if in reality, neighbouring characteristics share much less mutual information, it is reasonable that this could have an effect on the bias exhibited by a recruiter. Given that our experiment on modifying the application generation process showed that hiding information from recruiters had a moderate effect on the bias they exhibit, this concern is further supported. I should have verified that modifying the mutual information between characteristics in the applicant distribution doesn't affect the results of the experiments.

3. Characteristics are all categorical

This is not an assumption which holds. Realistically many features of a person are more accurately modelled as being continuous. However, it is unlikely to have a significant impact on the simulation. Any continuous characteristic can be represented to a degree of accuracy with a categorical characteristic.

4. An applicant's objective competence can be defined.

In real-world hiring, competence is subjective and contextual, but this reduction is a necessary one. We can only make limited conclusions on bias if we don't have some notion of meritoriousness to measure it in reference to.

5. Each applicant has only a single protected characteristic.

While this limits any insights I can draw on intersectionality, that is out of this dissertation's scope. Additionally, whether each characteristic is protected only affects the measurement of bias. So if this assumption doesn't hold, it would not affect the applicant distribution or the action of our recruiters.

6. **Applicants don't lie or omit information.**

This assumption won't hold in practice, but it is a functional simplification. It means I don't have to consider any bias introduced in these processes, so I can isolate the unfairness introduced by the ML models.

7. **Recruiters directly receive an applicant's competence, and receive a uniform random sample of applicants.**

As above, this assumption won't hold in practice. Any training set is likely to contain bias, and this will manifest as either the training set being a disproportionate sample of applicants or containing a systematic pattern of inaccuracies.

Instead, the training set is completely representative of the applicant distribution, that is, the training data is unbiased. As before, it allows us to remove another potential source of bias from the simulation. The only explanation for why bias could remain in the system is that it is somehow emergent in the action of the recruiters.

This means I have isolated the effect of algorithmic bias. But the research is now more limited; we can only obtain insights on this particular bias.

8. **Each recruiter makes a binary decision on each applicant**

This simplifies bias measurement, and in most instances, recruitment is binary. However, this may be overly reductive, for example, when an applicant may be offered a variable salary. Restricting project scope to binary classification is a standard practice in studies on machine fairness and intuitively these findings should generalise to the case of regression. However, there is a possibility that some bias may be exacerbated by constraining a recruiter to only one of two decisions, and I discuss this concern in Section 5.2.

9. **Equalised odds, demographic parity and the metrics I derived from them are valid measures of fairness**

I justified in Section 3.4 why these are suitable criteria for bias, while predictive parity isn't. At the start of the project, I constrained the scope of this research to output bias, that is, bias which can be measured entirely by measuring the decisions made by a recruiter. This approach may obscure some of the nuances within machine fairness. There are many opinions on how bias should be defined, and these include factors such as the intentionality of the classifier, the context in which the system is used, or the actual consequences the system has.

A.8 Tools Used for Writing Document

This document was generated with LaTeX,^[120] using the Fourier font^[121] and the Roboto font.^[122] The icons used in Figure 2.3.2 are from Bootstrap.^[123]

B Project Proposal

The project proposal begins on the subsequent page.

Using Bayesian Networks to Simulate Bias in Algorithmic Recruiting

The Candidate

October 2024

Supervisor - David Khachaturov (dgk27)

1 Introduction and Description

The use of automated decision-making software is becoming increasingly more prevalent in the field of recruiting^{[1][2]}, and there is a large body of evidence that these algorithms can exhibit bias in the decisions that they make^{[3][4][5]}.

Bias is poorly understood but very damaging, concentrating the negative consequences of inaccurate decisions onto a small group of people. In an inaccurate decision-making system, a person with a particular characteristic may be unable to get a job at a company because a pattern was identified and incorrectly trusted. In a biased decision-making system, a person with the same characteristics may be unable to get a job anywhere. That company will have no representation of people with those characteristics, and accumulated occurrences of these situations can further lead to harmful stereotypes or worse.

These concerns are being neglected. Already, many application-screening processes exist for which bias was only considered as an afterthought during development (examples are easy to find^[6]). To try to reduce the bias in these systems, training data may be modified or decisions passed through filters. For example, applicant names and sensitive characteristics are obscured^{[7][8]}, or groups which are known to be vulnerable to bias are given a compensatory adjustment^[9]. But without an understanding of what causes bias, the effectiveness of these measures cannot be adequately judged.

The dominant literary view is that bias in decision-making systems is caused by its propagation^{[10][11]}, either from the training data used by an algorithm or from its developers and designers. While this is a valid explanation for exhibited bias, it may be incomplete. Bias can originate in a system with unbiased training data and people. For example, a linear model which only had access to the school an applicant went to would be unable to base its decisions on anything else. If one school performed slightly better, then all the applicants the model proposes would be from that school. This bias cannot be explained by unrepresentative training data or predisposed designers. Rather, it is the structure and inputs of the algorithm itself.

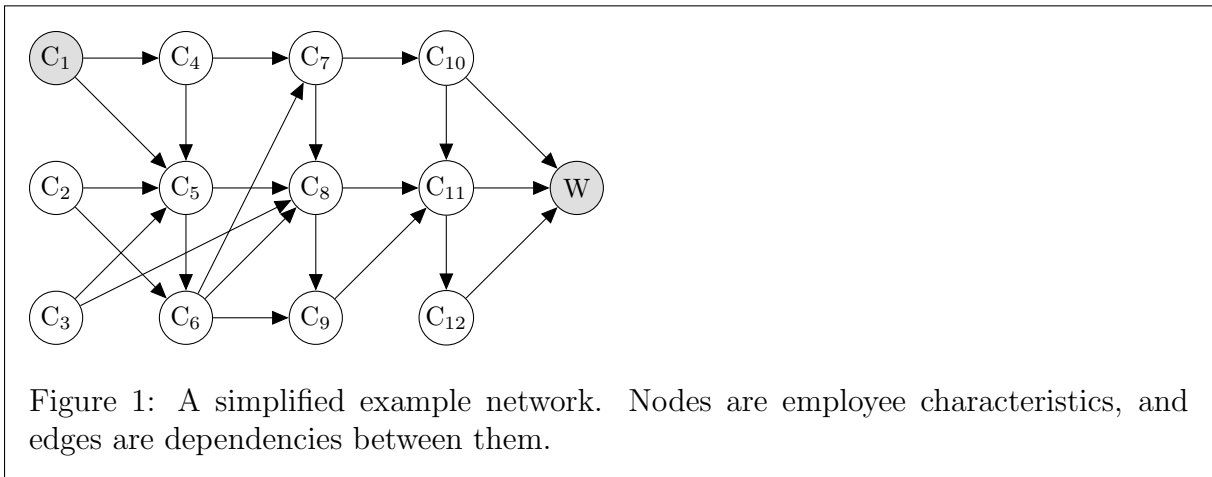
Therefore, I will explore if an algorithm's internal structure and inputs are predictive of bias. To do this, I will construct and justify a formal definition of bias and an adequate simulation in which it can be measured.

If this hypothesis were true, it would be important. If algorithmic recruiters are only biased because they propagate or amplify existing biases, it would suggest that they

aren't significantly worse than their human counterparts and that any damage can be fully mitigated by debiasing training data or counterbalancing known human biases. If instead, algorithmic recruiters are biased due to their very structure, these measures become ineffective. Bias will have to be carefully considered throughout the machine-learning process. Further, existing algorithms may be unequipped to make unbiased decisions in any context.

Simulating Algorithmic Recruiting

A set of applicants is modelled as a sample of a Bayesian network, where observed and hidden characteristics of each applicant probabilistically influence one another, as well as influencing a characteristic of that applicant's actual workplace suitability.



Algorithmic recruiters use the observed characteristics to predict an applicant's suitability. The suitability characteristic provides the ground truth for this simulation, and a recruiter can be measured by the deviation of its predictions from that ground truth. How an algorithmic recruiter is constructed is flexible, but at minimum, I will create a recruiter which uses a generalised linear model and one which uses a neural network. Linear models and neural networks are currently some of the dominant ideas within algorithmic recruiting^[12]. There is suggestion that simulating a Bayesian network may allow a recruiter to avoid some of the misleading patterns which can lead to biased thinking^[13].

To test the hypothesis, that an unbiased dataset may still produce a biased algorithm, the Bayesian network will be synthesised, rather than modelled empirically. This allows us to make the assumption that the data was collected in an unbiased manner and that the synthesised suitability is indeed the ground truth.

Some bias will exist in any real dataset, so this synthetic unbiased dataset would be impossible to collect empirically. Further, this means that the selection and distribution of the characteristics cannot be based on existing data or assumptions, which may be vulnerable to my own prejudices, or the prejudices of those who collected or generated the data. Therefore all characteristics in the dataset will be generic, rather than corresponding to some actual characteristic.

The network will include patterns and dependencies which mimic the abstract, collective patterns that link employees, without replicating individual dependencies or interactions. The point is that they codify patterns a machine learning model could use to make

predictions. Knowledge of what those patterns actually represent is unnecessary for the simulation.

One problem with this approach is that if a recruiter is identified to be prejudiced against a particular group, we are unable to determine whether it is biased against a protected characteristic, or a happenstance one. It is more societally damaging for a system to be biased against a particular race or gender rather than against a sporting preference, for example.

Regardless, this is a useful first step in determining how vulnerable a recruiter may be to bias.

For example, in Figure 1, C_1 could represent gender, which may be indirectly correlated to job suitability through factors like previous biased decisions or academic preferences. C_{10} , C_{11} , C_{12} could represent more closely correlated characteristics like interview score. A recruiter exposed to only C_1 may be biased.

Measuring Bias in the Simulation

The project must identify and justify why a chosen metric corresponds to the extent to which a recruiter exhibits bias. Suggested metrics include disparate impact^[14] and differential impact^[15].

Disparate Impact is measured by the ratio of the probability of one group receiving a positive outcome to the probability of another group receiving a positive outcome. Conventionally, one group will have a protected characteristic, and the other will be a control. While this may be a suitable measure for protected characteristics such as gender or race, it is inadequate because it does not account for real differences between the subsets. If older applicants are far more likely to be accepted, for example, it is difficult to know if this is the result of a biased system or just because those applicants are more suitable, for one of many possible reasons. (Chapter 3.1 of [16] offers additional justification for this viewpoint. Also see [17] for a case study on why being blind to sensitive characteristics may still be damaging).

Differential impact is equivalent to sensitivity (recall) except it is measured only for a particular group. Sensitivity is a model's ability to identify true positives (in this case, worthy applicants). The foundation is that if an algorithm's sensitivity to worthy applicants is low for a certain group, it can be said to be biased.

Other metrics exist, such as Predictive Parity (Group Benefit Equality), Demographic Parity, and Equalized odds^[18]. These metrics are mutually contradictory^[19] and present their own flaws. Further, better metrics may be produced. The advantage of an abstract simulation instead of an experiment is that we have access to the "ground truth", not just in terms of the actual suitability of an applicant, but all intermediary variables too.

Therefore, a combination of existing metrics and ones proposed in this dissertation will be used to evaluate the bias of a particular recruiter.

Exploration of the Simulation

Aspects of the simulation will be systematically varied, and how that affects the bias and accuracy of a particular recruiter will be measured.

Does a linear model recruiter have the potential to be more biased than a neural network

recruiter? Would a recruiter with access to information with a shorter causal distance from suitability exhibit a different level of bias? How does the act of varying features of the Bayesian Networks affect bias?

Evaluation

Evaluation of the project involves answering the following questions.

1. Is an algorithm's internal structure and inputs predictive of bias?
2. Does the simulation accurately reflect a real-world recruitment scenario?
3. Is the simulation useful?

Effect of Internal Structure and Inputs on Bias

The first question is amenable to scientific testing. It may be treated as a hypothesis. Across multiple randomised trials (such as varying the parameters of the Bayesian network), I investigate whether the recruiter's choice of algorithms (Linear Model, Neural Network, those created in the extension) are predictive of bias.

After gathering data from these trials, I conduct statistical testing to determine whether there are significant differences in the varying bias metrics.

Extension: I perform an additional test on whether input choice is predictive of bias, replicating the above experiment, while varying the inputs available to recruiters.

Real-world Applicability

The second and third questions are more challenging to quantify directly but were chosen because they best represent the goals of this project.

I critically analyse each assumption that was necessary to model the recruitment scenario in terms of how it is likely to affect the accuracy and generalisability of the simulation.

For example, I consider the fact that in this simulation, a recruiter is given direct access to employee characteristics, as opposed to indirectly approximating them through resumes and interviews. Whether this is likely to affect its validity is evaluated through reasoning.

I consider other potential hypotheses for any effect we might find. For example, if an effect were discovered between algorithmic structure and bias metrics, it would be reasonable to question if this effect is an increase in bias or a general decrease in accuracy. I consider the evidence for these alternative hypotheses.

Extension: Another assumption is that the dependencies between characteristics in the simulated network are representative of that of real empirical data approximated from employees. That is, that they are suitably complex and follow similar distributions. I will test whether this is a reasonable assumption by reviewing research, and determining whether Bayesian networks have had success in modelling real applicants and what distributions were discovered if they were.

Extension: I can investigate the generalisability of the simulation by testing whether it responds as expected to extreme examples. I test this by artificially increasing bias within

a subgroup through increasing the suitability metric that recruiters receive for individuals in a certain group. It is statistically tested whether the simulation detects this bias. I can create artificial recruiters designed to exhibit bias and verify that these are detected by the simulation. I can expose recruiters to all available information and verify that the simulation doesn't detect bias.

Usefulness of the Simulation

I discuss the implications of this research and the extent to which each of these implications could add value in algorithmic recruiting and beyond. I conduct a review of the research and practice in the field of algorithmic recruiting, determining what existing problems could be diagnosed by this project and whether it offers actionable solutions to those problems. Throughout, I must consider how the limitations of the simulation as discussed above limit its usefulness.

Extension: Another limiting factor to usefulness is the robustness of the bias metrics used. I test whether bias metrics are consistent under various conditions, and consistent with each other (for example, with Cohen's Kappa or correlation analysis).

2 Statement of Starting Point

I have written no code and I know little of the technical aspects of how to create such a simulation. For a talk I gave last year at Cambridge, I have done considerable research into Bayesian Networks, algorithmic recruitment, bias, and their interactions.

3 Criteria of Success

The project can be viewed as successful if the minimum following criteria are met (these are chosen to be measurable and so they are not comprehensive):

1. A simulation will be constructed which can generate samples from a Bayesian Network and send selected characteristics of these samples to algorithmic recruiters.
2. A subsystem to generate Bayesian networks which abstractly mimic the real dependencies which might occur in a recruitment scenario will be constructed.
3. Algorithmic recruiters will be constructed to predict the suitability of an applicant, based on observed characteristics of those applicants. One of those recruiters will be a generalised linear model and another will be a neural network.
4. The simulation will allow those recruiters to predict the suitability for applicants, and will be able to measure the extent to which they show bias.
5. A statistical test to determine whether the internal structure of a recruiter increases the amount of bias it shows within the assumptions made by the simulation will be conducted.

Extensions

1. Additional algorithmic recruiters will be constructed. One will construct a Bayesian network.

2. A second statistical test to determine whether the inputs given to a recruiter increases the amount of bias it shows within the assumptions made by the simulation will be conducted.
3. A research review and experiments on real resumes will be conducted to investigate whether the synthesised networks are sufficiently complex to model employee interactions.
4. The consistency of different metrics for bias will be investigated within the simulation.
5. The robustness of the simulation will be investigated by investigating whether artificially increasing the bias of recruiters is reflected in the simulation's evaluation of bias.

4 Plan of Work

Tools

The simulation will be developed with Python, using a library such as PyMC. A different language or software may be used to visualise the simulation. This isn't necessary, but may be useful to explore the simulation. GitHub will be used for version control.

Timings

1. **21st Oct - 4th Nov (will be spending a lot of time searching for a job here)** Set up a Github and Python environment. Decide on which Python libraries will be used and learn how they can be used to develop the proposed simulation. Develop a plan for the creation of the simulation, and start work on this.
Deliverable: Skeleton project showing file structure. Document with list of chosen libraries, justification, and experience gained. Small start to simulation in code.
2. **4th Nov - 18th Nov (entirety of first module is within this period, will still be job-hunting)** Develop the simulation (sc1) and plan the full structure of the codebase. Consider tools to visualise the simulation, and experiment with these.
Deliverable: Simulation in code with TODOs for network generation, recruiters, and bias metrics. Rough design document for structure of codebase. Rough plan of visualisation.
3. **18th Nov - 2nd Dec (end-of-term socialising, finalising job-hunting)** Develop bias metrics to be used and begin writing careful justification for this. Investigate and implement network generation (sc2). Finish the visualisation of the system. Finalise which recruiters will be generated.
Deliverable: Visualisation tool for visualising dependencies between characteristics, and which characteristics are observed. Network generation in code. Planning document of bias metrics to be used. Planning document of algorithmic recruiters to be generated, and how exactly they will function.
4. **2nd Dec - 20th Dec (holiday, but will spend some time working)** Develop a set of customisable recruiters (sc3) and begin experimenting with the full simulation. Conduct a risk assessment on how possible the extension criteria is to complete.

Deliverable: Algorithmic recruiters exist in code. Simulation is now functional except for bias metrics. Rough document detailing any interesting features discovered. Risk management document.

5. **20th Dec - 13th Jan (holidays, so will take a 3-week break here)** Slack time. Finish any outstanding work at this point, if the project is behind schedule.

Deliverable: No deliverables.

6. **13th Jan - 27th Jan (end of holidays, and second module preparation)** Implement measurement of the bias metrics (sc4).

Deliverable: Bias metrics exist in code. Document giving full justification for which bias metrics were used.

7. **27th Jan - 10th Feb (will still be recovering, finalising preparations for next year, accomodation etc.)** Continue to explore the simulation and write a more robust skeleton for the evaluation section. Complete statistical test on whether internal structure is predictive of bias (sc5). Consider the extension criteria. Plan and write progress report.

Deliverable: Evaluation section skeleton. Statistical test on whether internal structure is predictive of bias is conducted. Progress Report.

8. **10th Feb - 24th Feb (term begins, will do a lot of prep for second module and second module first project submission is due)** Consider extension criteria, if project is on schedule. Construct additional algorithmic recruiters (ec1) and conduct second statistical test (ec2). Develop first draft of evaluation section with TODOs for omissions.

Deliverable: Additional algorithmic recruiters. Justification and design document for second statistical test.

9. **24th Feb - 10th Mar** Structure the dissertation and write a skeleton draft. Complete the introduction section and preparation section. Compile all the resources used and research to be cited. Make progress on omissions in evaluation section and investigate different bias metrics for consistency (ec4).

Deliverable: Skeleton dissertation draft. Introduction and Preparation Section. Research collation document (research was collated throughout). More complete evaluation section. Investigation of different bias metrics.

10. **10th Mar - 24th Mar (second module submission)** Complete evaluation draft. Conduct research review and experiment on real resumes to investigate whether synthesised networks are adequately complex (ec3). Investigate robustness of simulation (ec5).

Deliverable: Evaluation draft.

11. **24th Mar - 7th Apr (holidays but will spend some time working)** Write full draft. Do final code review. Write risk assessment for any outstanding work.

Deliverable: Full dissertation draft with some omissions. Polished codebase.

12. **7th Apr - 21st Apr (holidays)** Refine dissertation and complete omissions. Continually iterate with supervisor.

Deliverable: Full dissertation.

13. **21st Apr - 5th May (end of holidays, start of third term)** Slack time, and time to complete any outstanding work.
14. **5th May - 12th May** Final refinement.
15. **Deliverable:** Final Submission

5 Resource Declaration

I intend to use my own computer and resources, with the MCS as a backup in case of failure. My work will be regularly backed up. All code I create will be regularly pushed to a GitHub repository, all my notes are automatically backed up, and I will maintain a separate GitHub for my progress on the dissertation document. I have two laptops and will be able to procure one quickly in the event they both break. I can otherwise use the MCS.

I accept full responsibility for this machine, and I have made contingency plans to protect myself against hardware and/or software failure.

References

- [1] Thakur A, Hinge P, Adhegaonkar V. Use of Artificial Intelligence (AI) in Recruitment and Selection. Social Science Research Network. Rochester, NY; 2023.
Available from:
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4452924
- [2] Ore O, Sposato M. Opportunities and risks of artificial intelligence in recruitment and selection. International Journal of Organizational Analysis. 2021 Jun 16; 30(6): 1771–82.
Available from:
https://www.researchgate.net/publication/352464056_Opportunities_and_risks_of_artificial_intelligence_in_recruitment_and_selection
- [3] Veldanda AK, Grob F, Thakur S, Pearce H, Tan B, Karri R, et al. Are Emily and Greg Still More Employable than Lakisha and Jamal? Investigating Algorithmic Hiring Bias in the Era of ChatGPT. arXiv.org. 2023.
Available from: <https://arxiv.org/abs/2310.05135>
- [4] Chen Z. Ethics and Discrimination in Artificial intelligence-enabled Recruitment Practices. Humanities and Social Sciences Communications. 2023 Sep 13;10(1):1–12.
Available from: <https://www.nature.com/articles/s41599-023-02079-x>
- [5] Chang X. Gender Bias in Hiring: An Analysis of the Impact of Amazon’s Recruiting Algorithm. Advances in Economics Management and Political Sciences. 2023 Sep 13;23(1):134–40.
Available from:
https://www.researchgate.net/publication/373896468_Gender_Bias_in_Hiring_An_Analysis_of_the_Impact_of_Amazon's_Recruiting_Algorithm

- [6] Kumar L, Penikalapati K, Gowrigari SKR. Resume Matching Framework via Ranking and Sorting Using NLP and Deep Learning. International Research Journal of Modernization in Engineering Technology and Science. 2023 Nov 24
Available from: https://www.researchgate.net/publication/382328627_RESUME_MATCHING_FRAMEWORK_VIA_RANKING_AND_SORTING_USING_NLP_AND_DEEP_LEARNING
- [7] Vivek R. Is Blind Recruitment an Effective Recruitment Method? International Journal of Applied Research in Business and Management . 2022 Dec;3(3):56–72.
Available from: https://www.researchgate.net/publication/366128032_Is_Blind_Recruitment_an_Effective_Recruitment_Method
- [8] Parasurama P, Sedoc J. Degendering Resumes for Fair Algorithmic Resume Screening. arXiv:211208910. 2022 Jul 12;
Available from: <https://arxiv.org/abs/2112.08910>
- [9] Deshpande KV, Pan S, Foulds JR. Mitigating Demographic Bias in AI-based Resume Filtering. Adjunct Publication of the 28th ACM Conference on User Modeling, Adaptation and Personalization. 2020 Jul 13;
Available from: <https://nlp-lab.umbc.edu/wp-content/uploads/sites/240/2020/08/FairUMAP.pdf>
- [10] "The study indicates that algorithmic bias stems from limited raw data sets and biased algorithm designers"^[4]
- [11] Belenguer L. AI bias: exploring discriminatory algorithmic decision-making models and the application of possible machine-centric solutions adapted from the pharmaceutical industry. AI and Ethics. 2022 Feb 10;2(2).
Available from: <https://link.springer.com/article/10.1007/s43681-022-00138-8>
- [12] Al-Quhfa H, Mothana A, Aljbri A, Song J. Enhancing Talent Recruitment in Business Intelligence Systems: A Comparative Analysis of Machine Learning Models. Analytics. 2024 Sep 1;3(3):297–317.
Available from: <https://www.mdpi.com/2813-2203/3/3/17>
- [13] Mecatti F, Vicard P, Musella F, Giammei L. Bayesian networks versus gender bias. Significance. 2022 Sep 26;19(5):16–20.
Available from: <https://academic.oup.com/jrssig/article/19/5/16/7072864>
- [14] Barocas S, Selbst AD. Big Data's Disparate Impact. California Law Review. 2016;104(3):671–732. Chapter II.
Available from: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2477899
- [15] Abràmoff MD, Tarver ME, Loyo-Berrios N, Trujillo S, Char D, Obermeyer Z, et al. Considerations for addressing bias in artificial intelligence for health equity. npj Digital Medicine. 2023 Sep 12;6(1):1–7.
Available from: <https://www.nature.com/articles/s41746-023-00913-9>

- [16] Dwork C, Hardt M, Pitassi T, Reingold O, Zemel R. Fairness Through Awareness. 2011.
Available from: <https://arxiv.org/pdf/1104.3913>
- [17] Skeem JL, Monahan J, Lowenkamp CT. Gender, Risk Assessment, and Sanctioning: The Cost of Treating Women Like Men. SSRN Electronic Journal. 2016;
Available from:
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2718460
- [18] Hardt M, Price E, Srebro N. Equality of Opportunity in Supervised Learning. 2016.
Available from: <https://arxiv.org/pdf/1610.02413>
- [19] Saravanakumar KK. The Impossibility Theorem of Machine Fairness – A Causal Perspective. arXiv:200706024. 2021 Jan 29;
Available from: <https://arxiv.org/abs/2007.06024>